

TABLE OF CONTENTS

Executive Summary	iv
List of Tables	v
List of Figures	vi
List of Abbreviations	vii
Symbols and Notations	viii
1 INTRODUCTION	1
1.1 Background	1
1.1.1 Speech Emotion Recognition as a Foundation	2
1.2 Motivation	2
1.3 Scope of the project	3
2 PROJECT DESCRIPTION AND GOALS	4
2.1 Literature review	4
2.1.1 Speech Emotion Recognition	4
2.1.2 Speech-Based Fatigue and Burnout Detection	5
2.1.3 Arousal-Based Approaches and Dimensional Emotion Modelling	6
2.1.4 Temporal Modelling of Affective States	7
2.2 Research Gap	8
2.3 Objectives	9
2.4 Problem statement	10
2.5 Project plan	11
3 TECHNICAL SPECIFICATION	12
3.1 Requirements	12
3.1.1 Functional Requirements	12
3.1.2 Non-Functional Requirements	12
3.2 Feasibility Study	13
3.2.1 Technical Feasibility	13
3.2.2 Economic Feasibility	13

3.2.3	Social Feasibility	13
3.3	System Specification	14
3.3.1	Hardware Specification	14
3.3.2	Software Specification	14
4	SYSTEM DESIGN	15
4.1	System Architecture	15
4.2	Detailed Design	16
4.2.1	Stage 1 Model Architecture	16
4.2.2	Stage 2 Temporal Model Architecture	17
4.2.3	Pseudo-Label Generation Module	18
5	METHODOLOGY AND TESTING	19
5.1	DATASETS	19
5.1.1	Emotion-to-Polarity Mapping	19
5.1.2	Class Imbalance Handling	20
5.2	AUDIO PREPROCESSING AND FEATURE EXTRACTION	21
5.2.1	Audio Preprocessing	21
5.2.2	Log-Mel Spectrogram Computation	21
5.2.3	Acoustic Feature Extraction for Arousal Computation	21
5.3	STAGE 1 - CRNN-BIGRU EMOTION CLASSIFIER	22
5.3.1	Input Representation: Log-Mel Spectrogram	23
5.3.2	Model Architecture	23
5.3.3	Training Configuration	25
5.4	STAGE 2 - AROUSAL-BASED PSEUDO-LABELLING AND TEM- PORAL MODELLING	27
5.4.1	Theoretical Grounding	27
5.4.2	Acoustic Arousal Score (A_{acoustic})	28
5.4.3	Emotion-Based Arousal Score (A_{emotion})	29
5.4.4	Composite Arousal Score (A_{final})	30
5.4.5	Fatigue Score (F_{score})	31
5.4.6	Pseudo-Label Generation via K-Means Clustering	31
5.4.7	Temporal Window Construction	34

5.4.8	BiGRU-Attention Temporal Fatigue Model	36
6	PROJECT IMPLEMENTATION	37
6.1	EXPERIMENTAL SETUP	37
6.2	STAGE 1 IMPLEMENTATION DETAILS	37
6.3	STAGE 2 IMPLEMENTATION DETAILS	38
6.4	ALPHA SENSITIVITY ANALYSIS IMPLEMENTATION	39
7	RESULTS AND DISCUSSION	40
7.1	STAGE 1: EMOTION CLASSIFICATION RESULTS	40
7.2	STAGE 2: FATIGUE AND BURNOUT DETECTION RESULTS . . .	42
7.3	ALPHA SENSITIVITY ANALYSIS	43
7.4	DISCUSSION	44
8	CONCLUSION AND FURTHER ENHANCEMENTS	45
8.1	CONCLUSION	45
8.2	FURTHER ENHANCEMENTS	46

EXECUTIVE SUMMARY

Fatigue and burnout are critical workforce health issues whose effects can be quantified in terms of workforce productivity, mental health, and overall physical health. Although it is becoming more common, automated detection of these conditions has not been reliable because, in part, standard speech datasets do not have any fatigue-specific annotations. The current work fills this gap by suggesting a completely automated, annotation-free two-stage pipeline to identify fatigue and early burnout using raw speech signals only. It is trained on the Log-Mel spectrogram of three publicly available acted-speech corpora CREMA-D, RAVDESS and TESS that contain about 11,682 utterances of a wide variety of speakers.

The model splits speech into three high-level polarity categories - Positive, Negative, and Neutral - with a total accuracy of 93.00 % and a weighted F1-score of 93.06. The phase forms the basis of an arousal estimation process that is principled and theoretically based. The second stage uses probabilities of emotion classes generated by Stage 1 along with six hand engineered acoustic features RMS energy, fundamental frequency mean and variance, spectral centroid, spectral rolloff and zero crossing rate to compute a composite arousal score per utterance.

The arousal-based calculation is based on the Circumplex Model of Affect developed by Russell and the obtained fatigue scores are fitted to the Maslach Burnout Inventory paradigm. The fatigue score distribution is then clustered using K-Means to produce pseudo-labels of three levels of fatigue: Non-Fatigued, Mild Fatigue, and High Fatigue/Burnout. A ten utterance sliding window per speaker drives a BiGRU architecture with Scaled Dot-Product Attention, aiding the gradual degradation of arousal that is a hallmark of natural burnout progression. Five-fold stratified cross-validation results in a mean accuracy of 90.16 % and a mean Macro F1-score of 90.45 % with the highest fold having 94.51 % accuracy and 94.65 % F1. The suggested system does not require any fatigue-related annotations, is speaker-independent, and it is suitable to be used in unobtrusive, real-time workplace monitoring.

LIST OF TABLES

2.1	Project Plan and Milestones	11
3.1	Hardware Specification	14
3.2	Software Specification	14
5.1	Overview of speech corpora used for emotion model training.	19
5.2	Emotion-to-polarity mapping and arousal weights.	20
5.3	Acoustic features extracted for Stage 2 arousal computation.	22
5.4	Layer-wise architecture summary of the Stage 1 CRNN-BiGRU model.	25
5.5	Stage 1 training hyperparameters.	26
5.6	Acoustic feature weights for A_{acoustic} computation.	29
5.7	K-Means cluster centroids and pseudo-label assignments.	32
5.8	Feature channels in each temporal window timestep vector.	35
7.1	Stage 1 emotion classification results on the full combined dataset.	40
7.2	Five-fold stratified cross-validation results for Stage 2 fatigue detection.	42

LIST OF FIGURES

4.1	End-to-end pipeline architecture of the proposed fatigue detection system.	15
4.2	CRNN-BiGRU model architecture for Stage 1 emotion classification.	16
4.3	Temporal window construction and BiGRU-Attention model architecture for fatigue prediction.	17
5.1	Arousal-based fatigue score computation and pseudo-label generation workflow.	33
5.2	F-Score distribution with K-Means cluster boundaries.	33
6.1	Training and validation loss and accuracy curves — Stage 1 CRNN-BiGRU Emotion Classifier (Best Fold).	38
6.2	Training and validation loss and accuracy curves — Stage 2 BiGRU-Attention Fatigue Model (Best Fold).	39
7.1	Confusion Matrix — Stage 1 CRNN-BiGRU Emotion Classification Model.	41
7.2	Confusion Matrix — Stage 2 BiGRU-Attention Fatigue Detection Model (Best Fold).	42
7.3	Alpha Sensitivity Analysis - Macro F1 across arousal fusion weight values.	44

LIST OF ABBREVIATIONS

SER	Speech Emotion Recognition
CRNN	Convolutional Recurrent Neural Network
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
GRU	Gated Recurrent Unit
BiGRU	Bidirectional Gated Recurrent Unit
LSTM	Long Short-Term Memory
STFT	Short-Time Fourier Transform
MFCC	Mel-Frequency Cepstral Coefficients
RMS	Root Mean Square
ZCR	Zero Crossing Rate
FFT	Fast Fourier Transform
GPU	Graphics Processing Unit
CUDA	Compute Unified Device Architecture
cuDNN	CUDA Deep Neural Network Library
WHO	World Health Organization
MBI	Maslach Burnout Inventory
SUSAS	Speech Under Simulated and Actual Stress
AVEC	Audio/Visual Emotion Challenge
ComPARE	Computational Paralinguistics Challenge

SYMBOLS AND NOTATIONS

Symbol	Description
$A_{acoustic}$	Acoustic arousal score
$A_{emotion}$	Emotion-based arousal score
A_{final}	Composite arousal score
F_{score}	Fatigue score
$P(class_k)$	Probability of class k
w_k	Weight of class k
N_k	Number of samples in class k
$S_{mel}(t, m)$	Log-Mel spectrogram value
$H_{mel}(m)$	Mel filterbank function
ϵ	Small constant for numerical stability
h_t	Hidden state at time step t
z_k	Logit of class k
α	Arousal fusion hyperparameter
B	Batch size
T	Number of time steps
F_0	Fundamental frequency

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Never before has occupational stress-related disorders been on the increase in the modern workplace, where burnout and chronic fatigue have become two of the most widespread and cost-effective problems. Occupational burnout was officially recognized as a syndrome by the World Health Organization as a consequence of unmanaged, chronic stress at work, defined in terms of three dimensions: energy depletion, work-related mental detachment, and work-related professional ineffectiveness. In spite of this awareness, early and automatic identification of these conditions is still an unsolved problem, mostly due to the fact that the physiological and affective alterations that go along with fatigue are not discrete phenomena that can be observed but are a gradual change as time goes by.

The conventional methods of diagnosing are based on self-administered questionnaires like the Maslach Burnout Inventory, clinical interviews, or physiological monitoring gadgets. Although these are validated methods of clinical understanding, both types have constraints in practice: questionnaires require valid introspection, which can hardly be given by exhausted individuals, and physiological sensors are costly, invasive, and impractical to implement as continuous and uncontrolled. This thus creates a strong need to have passive, non-invasive methods which are capable of identifying burnout indications early on due to naturalistic behaviour signs.

It is a well-known fact that speech is one of the richest non-invasive cues of affective and physiological conditions. Voice acoustic features such as fundamental frequency (F0), vocal energy, speech rate, and spectral distribution vary in a systematic manner in response to arousal, emotional valence, and physical fatigue. Psychoacoustic and computational paralinguistic studies have repeatedly shown that fatigued speech is less variable in F0, has reduced RMS energy, is slower-paced and has a negative affect towards low-arousal. These features render speech a perfect candidate in passive, continuous occupational health monitoring.

1.1.1 Speech Emotion Recognition as a Foundation

The current state of Speech Emotion Recognition (SER) has come a long way in the past ten years; the current deep learning models have reached high accuracy on common emotional speech datasets. But direct mapping of SER capabilities to fatigue detection is not a trivial task. Fatigue is not annotated in standard emotion corpora, fatigue has acoustic similarities to low-arousal emotional states like sadness and boredom, and burnout is a long-term process that cannot be measured in one utterance. The work fills these gaps by considering SER a foundation layer and developing a new arousal-based temporal modeling model over it.

1.2 MOTIVATION

The accumulating body of data that has shown a strong relationship between chronic fatigue in the workplace and not only the lack of productivity in an organization, but also the poor health of individuals offers a strong incentive to the creation of automated early-warning protocols. Occupational medicine research indicates that burnout is a continuous process, starting with mild emotional fatigue and progressing to severe functional impairment in weeks or months. This slow property has made it susceptible to longitudinal signal analysis - but current automated methods in speech-based affective computing have been predominantly concerned with discrete emotion recognition in the short term as opposed to state estimation over long periods, which is health-relevant.

The current study is motivated by the fact that the acoustic indicators of burnout, specifically arousal depletion and the persistent negative affective expression can be identified in regular emotion speech data without fatigue-specific labels. This project closes the gap in research between existing SER methods and clinically relevant prediction of burnout by framing fatigue detection as an application of arousal estimation. The rationale is a wish to develop a system that is practical, deployable, and reproducible solely using publicly available data and open-source applications.

Moreover, the growing use of remote and hybrid work models, along with the constraints posed by conventional burnout measurement methods in non-clinical contexts, highlights the pressing need to create lightweight, annotating-free monitoring solutions.

This pipeline is cost-effective and scalable because the proposed pipeline does not require additional human labelling effort, other than that already in existing emotion datasets.

1.3 SCOPE OF THE PROJECT

The present project scope includes the design, implementation, and empirical testing of a two-stage speech analysis pipeline in predicting fatigue and burnout. The system does not need video, physiological sensors, or text transcriptions and is only audio-based. The former stage deals with the issue of speech emotion recognition with the help of a hybrid deep learning architecture; the latter stage complements this recognition with a fatigue severity prediction problem with theoretically based arousal calculation and time modelling.

The following scope considerations delimit the project. The system is also trained and tested on acted-speech corpora -CREMA-D, RAVDESS, and TESS - and although these corpora offer controlled and well-labeled emotional data, do not directly reflect naturalistic workplace speech. The future work is discussed as generalization to real-life situations. The pipeline concentrates on three severity levels of fatigue: Non-Fatigued, Mild Fatigue, and High Fatigue/Burnout following the conceptual framework of the Maslach Burnout Inventory.

Other extensions that are not yet covered are speaker verification and individual baseline calibration though they are recognized to be important. The system is designed to be speaker-independent, i.e. no per-speaker adaptation is performed in inference. Each and every element, such as audio preprocessing to temporal fatigue classification, is in Python utilizing open-source libraries that are readily available, and hence reproducible and accessible to the research community.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

The three broad fields considered in the literature that are relevant to this project are: speech emotion recognition (SER), speech-based fatigue and burnout detection, and arousal-based temporal modelling of affective states. All these regions have grown significantly in the last 20 years, and their intersecting point is the conceptual base of the proposed pipeline.

2.1.1 Speech Emotion Recognition

Speech Emotion Recognition has a long and well-documented history in the computational paralinguistics community. Initial methods were based on manually constructed acoustic characteristics - mel-frequency cepstral coefficients (MFCCs), pitch contours, energy statistics, and formant frequencies - computed over small analysis windows and input to conventional classifiers like Support Vector Machines, Hidden Markov Models, and Gaussian Mixture Models. Schuller et al. (2009) developed powerful benchmark evaluation schemes with the INTERSPEECH Emotion Challenges, which standardized feature sets and evaluation protocols, allowing systematic advances in the field. Deep learning offered a tremendous performance boost to SER.

Convolutional Neural Networks were found to be as capable of extracting local spectro-temporal patterns of spectrogram representations, whereas Recurrent Neural Networks, especially LSTMs and GRUs, were found to be reasonable at modeling the temporal dynamics of emotional speech. Cho et al. (2014) proposed a computationally efficient variant of the LSTM, the Gated Recurrent Unit, with similar sequence modelling ability using fewer parameters. The bi-directional extension of GRUs enables models to use context of the past and future in an utterance, which is especially useful in utterance-level classification tasks where the emotional content is spread over the entire signal duration.

Hybrid CRNN models are convolutional feature extractors and recurrent sequence modellers, which have demonstrated state-of-the-art performance on standard SER benchmarks. As Trigeorgis et al. (2016) showed, end-to-end learned features based on raw waveforms, transformed by deep convolutional and recurrent layers, can be more effective than handcrafted feature pipelines. This observation encourages the use of CRNN-BiGRU architecture in the initial phase of the current study, where the input representation is that of Log-Mel spectrograms, a trade-off between interpretability of known acoustic characteristics and the discriminatory capability of deep learned representations.

Recent developments in SER have included transformer-based designs and self-trained models like wav2vec 2.0 and HuBERT. These methods use large-scale pre-training on unlabelled audio to train general-purpose representations of speech, which are transferrable to downstream classification tasks. It has been demonstrated by Pepino et al. (2021) and other authors that emotion datasets fine-tuning of pre-trained speech transformers outperforms CNN-RNN baselines, especially in low-resource conditions. Although these methods are not utilized in the current work - since they require more computational time and are less interpretable - they are a valuable direction in the future to enhance performance.

2.1.2 Speech-Based Fatigue and Burnout Detection

In contrast to emotion recognition, the research on speech-based fatigue and burnout identification is much less advanced, due to the lack of relevantly labelled data and the difficulty of identifying fatigue as a computation object. Initial research by Greeley et al. (2006) on the SUSAS corpus, showed that speech of fatigued and stressed individuals could be identified as opposed to neutral speech on a basis of objective differences in energy and F0 variability, which offered an early empirical foundation to estimate acoustic fatigue.

Han et al. (2020) explored the concept of speech-based analysis during the COVID-19 environment and explored the correlations between speech features and self-reported fatigue, sleep quality, and anxiety. Their results supported their correlation between enduring negative affect in speech and emotional depletion states, which is conceptually

similar to burnout. Similar studies on depression detection, especially on the use of the DAIC-WOZ corpus, have also revealed that speech-based longitudinal negative affect is a valid predictor of clinically meaningful affective and physical exhaustion.

A continuous speech-based fatigue detection system was suggested by Vekkot et al. (2025), showing that the implementation of acoustic fatigue monitoring in the safety-related workplace is practically viable. They applied a mixture of acoustic characteristics and recurrent sequence models in their work to monitor fatigue accumulation over long periods of monitoring, proving the significance of temporal modelling of burnout-related tasks. Likewise, Gaballah et al. (2021) used Bi-LSTM classifiers to detect stress in hospital workers based on context, highlighting the importance of bidirectional recurrent architectures on longitudinal health monitoring.

One of the endemic weaknesses of previous research in this area is the use of specially curated fatigue corpora or clinical annotations, which are costly to generate and hard to extrapolate to populations. Kumar et al. (2024) and Pradhan et al. (2024) suggested deep learning methods to detect psychological stress and workplace burnout respectively, yet both methods needed specific labelled data related to domains, limiting their applicability. The current work breaks this paradigm by creating fatigue pseudo-labels using a normal emotion dataset based on an arousal-based intermediate representation, allowing the utilization of existent, large scale speech corpora without any extra annotation costs.

2.1.3 Arousal-Based Approaches and Dimensional Emotion Modelling

Circumplex Model of Affect, suggested by Russell (1980) is a theoretically motivated two-dimensional model of how to structure emotional states in terms of valence (positive/negative) and arousal (high/low energy). This dimensional viewpoint provides a natural point of contact between discrete taxonomies of emotion and the graded nature of fatigue as fatigue can be operationalized as a state of low long-term arousal and negative or neutral affective expression.

Goudbeek and Scherer (2010) carried out a systematic study of the acoustic correlates of the valence and arousal dimensions with vocal energy and fundamental frequency being the most notable and valid acoustic indicators of arousal. This observation guides

the weighting scheme of features used in the arousal computation module of Stage 2, with the greatest weights in the composite arousal score formula going to RMS energy and F0 measures. The arousal weights attributed to each of the polarity classes of emotions used in the current work are based directly on the circumplex geometry proposed by Russell that offers a clear and replicable labelling structure.

Schuller et al. (2013) further generalized computational paralinguistics benchmark framework to dimensional affect estimation and showed that model-predicted arousal scores are predictive of a broad spectrum of speaker states beyond discrete emotions, such as health conditions and cognitive load. This work standardized the ComPARE feature set as a standard of reference in paralinguistic feature extraction and empirically validated using arousal as a mediating representation between emotion and prediction of health state.

Later studies have investigated how pre-trained dimensional emotion models can be utilized in downstream health-related speech analysis. Ringeval et al. (2019) also showed that arousal estimates based on the AVEC challenge benchmark corpora can be used to predict depression severity, which also justified the use of arousal as an intermediate representation to estimate fatigue. The current study operates on the basis of this conceptual framework by utilizing arousal as a pseudo-label generation process and a channel of input features into the temporal fatigue classification model.

2.1.4 Temporal Modelling of Affective States

The time aspect of emotional and health related conditions have gained more and more attention in the study of affective computing. The dynamics of emotional expression over time have been captured by sliding window techniques, recurrent architecture and attention mechanisms. Gaballah et al. (2021) have shown that context-aware Bi-LSTM classifiers, which are trained on sequences of acoustic feature vectors of back-to-back speech segments, are highly effective in performing stress detection tasks compared to utterance-level classifiers, which is in line with the significance of multi-utterance context.

Attention mechanism, as proposed by Bahdanau et al. (2015) in correlation with neural machine translation and later generalized to the Scaled Dot-Product Attention formulation by Vaswani et al. (2017) allows sequence models to weigh the time steps of a sequence selectively depending on their importance to the prediction task. In the context of burnout detection, this is especially useful since the acoustic cues of fatigue may not be evenly distributed over the utterances of a speaker - some time steps can be more burnout-relevant than others and the attention mechanism can enable the model to detect and emphasize such informative steps.

These findings directly influence the choice of BiGRU sequence modelling coupled with Scaled Dot-Product Attention that is implemented in Stage 2 of the current work. The model's design, using a window of ten utterances per speaker and an arousal trend slope as an explicit temporal predictor, aims to capture the cumulative, linear nature of burnout accumulation as opposed to responding to discrete low-arousal utterances, which could be due to temporary emotional responses and not necessarily exhaustion.

2.2 RESEARCH GAP

Although the above-discussed literature has recorded significant advancements, a number of gaps are still important that are to be filled in the current work. To begin with, most speech-based fatigue detection systems are based on domain-specific annotated corpora or clinical labels, which are costly to generate and lack generalizability. The lack of non-proprietary, massively annotated fatigue-tagged speech datasets has severely limited the training of effective and repeatable detection systems. The current study fills this gap by suggesting an annotation-free pseudo-labelling procedure that produces fatigue labels based on normal emotion corpora with theoretically-grounded arousal computation.

Second, the current strategies have focused on fatigue predominantly as an utterance-level phenomenon without taking into consideration the longitudinal and cumulative progression of burnout. The gradual arousal depletion that is characteristic of true burnout and not just temporary low-arousal states like sadness or boredom cannot be described by single-utterance classifiers. The sliding window time model of Stage 2

of this paper specifically features this limitation by combining information over ten successive utterances and adding an explicit feature of arousal trend slope.

Third, the connection between discrete classification of emotions and continuous estimation of fatigue has not been exhaustively used in the literature. Typically, most burnout detection systems are based on unprocessed acoustic features, or the presence of pre-trained dimensional models, without utilizing the rich discriminative information of emotion class probabilities. The current paper addresses this gap by considering the output probabilities of a trained emotion classifier as an interpretable, principled input to the arousal estimation module.

Lastly, the arousal-based pseudo-labelling sensitivity to the fusion hyperparameter has not been properly analyzed in the literature. In this work, the alpha sensitivity analysis offers an empirical characterization of the reaction of the composite arousal score formula to change in the emotion-acoustic fusion weight in a systematic way, which leads to the reproducibility and strength of the proposed methodology.

2.3 OBJECTIVES

The main aims of this project are the following:

1. To train a strong three-class speech polarity classifier (Positive, Negative, Neutral) with a CRNN-BiGRU architecture that has been trained on the Log-Mel Spectrograms of various acted-speech corpora.
2. To develop a theoretically-based, annotation-free arousal computer model, which integrates acoustic attributes and emotion likelihoods according to the Circumplex Model of Russell.
3. To generate fatigue pseudo-labels of typical emotion speech data sets with K-Means clusters based on composite fatigue scores of sustained negative affect and arousal depletion.
4. To construct and test a temporal BiGRU model with Scaled Dot-Product Attention classifying cumulative fatigue between ten-utterance windows of speakers into three severity levels.

5. To perform intensive analysis with 5-fold stratified cross-validation and sensitivity analysis of the arousal fusion hyperparameter α , to show model robustness.
6. To offer a speaker-independent, annotation-free pipeline that can be applied to the reality of fatigue monitoring in workplaces without the need of special equipment or fatigue annotations.

2.4 PROBLEM STATEMENT

Occupational fatigue and early burnout are clinically important diseases, which are gained during the course of time during emotional and physiological stress. Burnout is officially defined by the WHO as an occupational phenomenon characterized by emotional exhaustion, depersonalization and low personal efficacy, which when unnoticed has serious psychiatric and cardiovascular consequences.

These states are encoded in speech by systematic acoustic modifications - decreased fundamental frequency variability, decreased vocal energy, and a gradual affective dislocation but there is no reliable way to automatically identify fatigue in speech without fatigue-tagged corpora or invasive physiological devices. It is these changes that are a trend throughout the longer interactions and thus fatigue is inherently temporal and cumulative and not identifiable in any one utterance.

The essence of the technical problem is to develop a two stage pipeline that can: (1) classify speech by emotional polarity with deep learning on conventional emotion corpora, (2) estimate arousal in a principled, theoretically-informed way through acoustic features and emotion class probabilities, (3) simulate fatigue severity pseudo-labels without human annotation, and (4) model arousal depletion across consecutive utterances to distinguish genuine burnout from transient low-arousal states - by reconciling computational speech emotion modeling, Russell's Circumplex Model, and the Maslach Burnout Inventory.

2.5 PROJECT PLAN

Table 2.1: Project Plan and Milestones

Phase	Activities	Duration	Deliverable
Phase 1: Literature Review & Design	Survey of SER, fatigue detection, arousal modeling, and temporal sequence literature; identification of research gap; system architecture design	2 Weeks	Literature review table; system design document
Phase 2: Data Preparation	Dataset acquisition (CREMA-D, RAVDESS, TESS); audio preprocessing pipeline; Log-Mel spectrogram extraction; acoustic feature computation; class imbalance analysis	2 Weeks	Preprocessed dataset; feature tensors; train/-val/test splits
Phase 3: Stage 1 - Emotion Classifier	CRNN-BiGRU model implementation; training with WeightedRandomSampler and class-weighted loss; 5-fold cross-validation; performance evaluation	3 Weeks	Trained emotion classifier (93.00% accuracy); confusion matrix; classification report
Phase 4: Stage 2 - Pseudo-labeling & Fatigue Scoring	Acoustic arousal score computation; emotion-based arousal score fusion; composite arousal formulation; K-Means pseudo-label generation; fatigue score distribution analysis	2 Weeks	Fatigue pseudo-labels for full dataset; cluster centroid analysis; F-score distribution plot
Phase 5: Stage 2 - Temporal Fatigue Model	Sliding window construction; temporal feature tensor assembly; BiGRU-Attention model training; 5-fold cross-validation; alpha sensitivity analysis	3 Weeks	Trained temporal model (90.45% mean Macro F1); fold-wise results; sensitivity plot
Phase 6: Evaluation & Reporting	Comprehensive evaluation; confusion matrices; training curves; result analysis; report preparation and documentation	2 Weeks	Final project report; conference paper; all evaluation artifacts

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 REQUIREMENTS

3.1.1 Functional Requirements

The system will receive raw audio files as input and will give a three-class fatigue severity outcome (Non-Fatigued, Mild Fatigue, High Fatigue/Burnout) to a particular set of utterances produced by an individual speaker. The preprocessing module should enable conversion of audio files into 16 kHz mono, reduction of silence, normalization of loudness and padding/truncation to 3 seconds. The Stage 1 module should be able to recognize 80-band Log-Mel spectrograms and categorize each utterance into one of three classes of polarities with probability scores. The Stage 2 module has to identify six scalar acoustic features per utterance, compute composite arousal scores, create pseudo-labels with the help of K-Means clustering, and categorize ten utterance temporal windows into fatigue severity levels.

3.1.2 Non-Functional Requirements

The system should be speaker-independent, and need no per-user calibration or enrollment. Each element should be deployed on open-source Python libraries to make them reproducible and accessible. The training pipeline should be able to do five fold stratified cross-validation using a fixed random seed (SEED=42) to ensure that the results are reproducible between experimental runs. The system should deal with class imbalance by using both WeightedRandomSampler and class-weighted loss functions, and making sure that minority fatigue classes are not systematically under-represented in the trained models.

3.2 FEASIBILITY STUDY

3.2.1 Technical Feasibility

The proposed pipeline is technically feasible using current open-source deep learning frameworks and publicly available datasets. PyTorch has efficient implementations of all the necessary neural network elements, such as convolutional layers, GRU cells, bidirectional wrappers, and attention mechanisms, which are accelerated by the GPU. Well-validated implementations of Log-Mel spectrogram extraction and the acoustic feature extraction functions needed in Stage 2 are available in the librosa library. Google Colab offers enough GPU compute resources to train both stages within realistic time limits, where Stage 1 training can take around 2-3 hours per fold, and Stage 2 training can take less than an hour per fold.

3.2.2 Economic Feasibility

The datasets, libraries and computational resources utilized in this project are either free or provided at no cost on academic licenses. CREMA-D, RAVDESS and TESS are open-source researched. Scikit-learn, pyTorch, librosa, and others are open-source. The free version of Google Colab offers access to a GPU that is adequate to support the needs of this project in terms of training. No special hardware investment is thus necessary in the proposed system and it can be re-created by any researcher with access to common academic computing facilities.

3.2.3 Social Feasibility

The introduction of non-invasive speech-based monitoring systems in the workplace evokes some critical issues of privacy, consent and data control. The current system will only work on the audio basis and will not need the availability of video, location data, or personally identifying information. In a production deployment, proper informed consent processes, data anonymization policies, and access control policies would have to be implemented. It is intended to be an early-warning system, to be used in organizational wellbeing programmes, and not as a surveillance system, and its outputs should lead to supportive measures and not punitive measures.

3.3 SYSTEM SPECIFICATION

3.3.1 Hardware Specification

Table 3.1: Hardware Specification

Component	Specification
Processor	Intel Core i5 or higher (training: NVIDIA GPU via Google Colab T4/V100)
RAM	Minimum 8 GB (16 GB recommended for full dataset processing)
Storage	Minimum 10 GB for datasets, spectrograms, model checkpoints, and outputs
Audio Input (Deployment)	Standard microphone (16-bit, 16 kHz sampling rate)
GPU (Training)	NVIDIA Tesla T4 (Google Colab) — CUDA 11.x, cuDNN 8.x
Operating System	Ubuntu 20.04 LTS / Windows 10 (Python 3.8+ compatible)

3.3.2 Software Specification

Table 3.2: Software Specification

Library / Tool	Version	Purpose
Python	3.9+	Primary programming language
PyTorch	2.0+	Deep learning framework for CRNN-BiGRU and BiGRU-Attention models
librosa	0.10+	Audio loading, preprocessing, Log-Mel spectrogram and acoustic feature extraction
NumPy / SciPy	Latest stable	Numerical computing; linear regression for arousal trend slope
scikit-learn	1.3+	K-Means clustering, stratified K-Fold, classification metrics, normalization
Matplotlib / Seaborn	Latest stable	Visualization of training curves, confusion matrices, F-score distributions
Google Colab	N/A (cloud)	GPU-accelerated training environment
torchaudio	2.0+	Audio dataset loading and on-the-fly augmentation

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

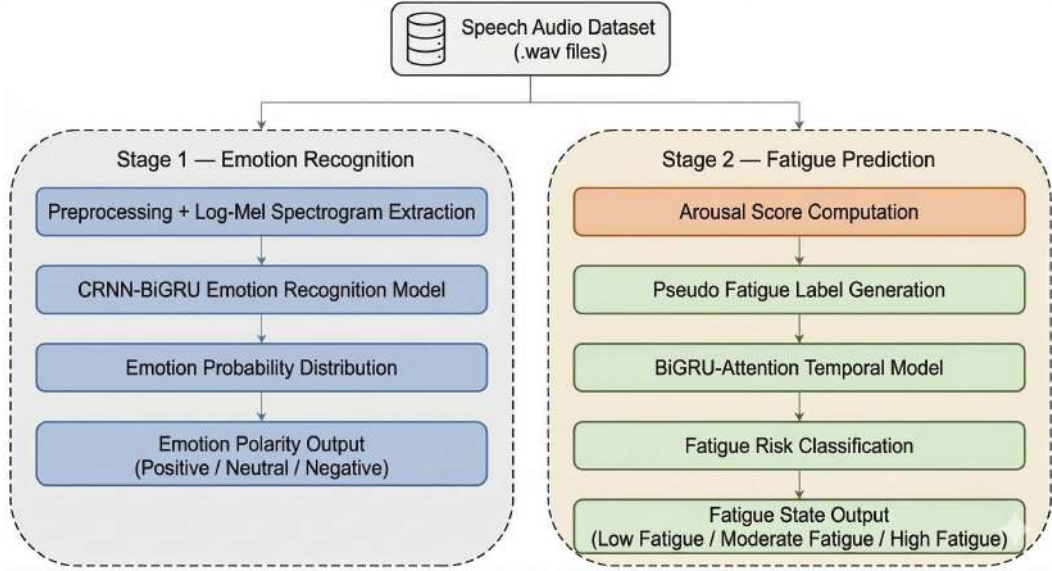


Figure 4.1: End-to-end pipeline architecture of the proposed fatigue detection system.

The general system architecture is a two-stage sequential pipeline design. The former level is the utterance level, and it takes a Log-Mel spectrogram as input and emits a three-dimensional probability vector with the polarity categories being Positive, Negative and Neutral. The second step merges utterance-level data over a ten utterance sliding window of the same speaker with emotion probabilities and acoustic features to form a composite arousal representation which is fed to a temporal BiGRU-Attention classifier.

The flow of data will look like this: raw audio files are fed into the preprocessing module, where they are converted into different formats, stripped of silence, loudness normalized, and fixed length padded. The preprocessed waveforms are sent in parallel to two branches: spectrogram extraction branch, which computes the (80, 301) Log-Mel feature tensor that is fed to Stage 1, and the acoustic feature extraction branch, which computes the six scalar features that are fed to Stage 2 arousal computation. Stage 1 treats individual utterances in isolation and Stage 2 treats sequences of utterances

organized in time and by speaker.

This architectural isolation of concerns between utterance-level emotion classification and window-level fatigue prediction gives modularity and theoretical interpretability. The stages are individually verifiable and the stage of arousal in between gives a verifiable explanation of the fatigue projections of stage 2.

4.2 DETAILED DESIGN

4.2.1 Stage 1 Model Architecture

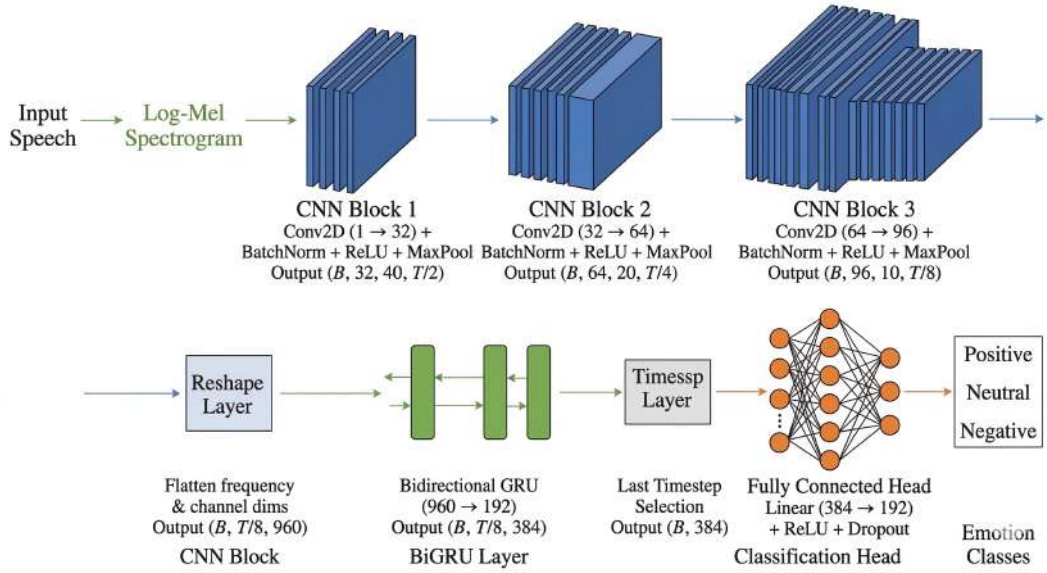


Figure 4.2: CRNN-BiGRU model architecture for Stage 1 emotion classification.

The Stage 1 emotion classifier is constructed on top of a Convolutional Recurrent Neural Network (CRNN) having a Bidirectional GRU sequence layer. The architectural philosophy is to extract local spectro-temporal patterns with CNN component and extract global temporal patterns across the entire utterance duration with BiGRU component.

The CNN block includes three consecutive convolutional blocks, which include Conv2D (kernel size 3×3 same padding), BatchNorm2D, ReLU activation and Max-Pool2D (2×2). The three stages increase the number of feature maps (32, 64, 96) which allows the network to extractively describe successively abstract spectro-temporal fea-

tures. The three pooling operations decrease frequency and time dimensions by a factor of 8.

A sequence of $(B, T/8, 960)$ feature vectors - with frequency and channel dimensions flattened - is input to the BiGRU layer, which outputs a bidirectional hidden state tensor of size $(B, T/8, 384)$. The last time step is removed as the utterance-level representation, which is fed through a two-layer fully connected classification head containing ReLU activation and 0.3 dropout between layers, and finally giving three-class logits which are converted to probabilities using softmax.

4.2.2 Stage 2 Temporal Model Architecture

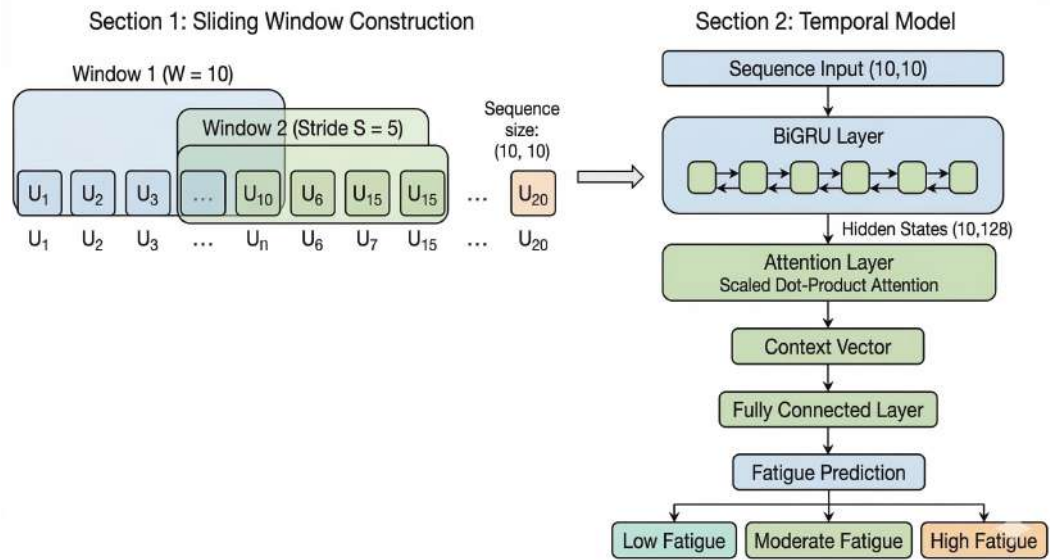


Figure 4.3: Temporal window construction and BiGRU-Attention model architecture for fatigue prediction.

The Stage 2 temporal fatigue model is a Bidirectional GRU with Scaled Dot-Product Attention which processes 10-step lists of 10-dimensional feature vectors. Every feature vector at each timestep represents six normalized acoustic features, emotion-based arousal score, composite arousal score, utterance-level fatigue score, and arousal trend slope - all of which form a rich multimodal representation of the fatigue-related features of each utterance.

The 10-step sequence is fed through the BiGRU, using a hidden size of 64 and 2 layers with inter-layer dropout of 0.3, resulting in hidden states of size (B, 10, 128). The attention module implements the scaled dot-product attention based on the 10 time steps, calculating the query, key, and value projections and obtaining the softmax-normalized attention weights that enable the model to selectively focus on the best utterances that can be the most indicative of the fatigue state of the speaker in the window. The context vector weighted by attention is then forwarded through the last classification head.

4.2.3 Pseudo-Label Generation Module

The pseudo-label generation module works on the entire dataset F-score distribution. The fatigue score is calculated based on the composite arousal score and the negative class probability per utterance based on the weighted equation that is detailed in the methodology chapter. K-Means clustering ($k=3$) is used on the whole F-score distribution and each utterance is labelled with the nearest centroid of a cluster. The cluster assignment is deterministic with respect to the fixed K-Means random state such that the identical pseudo-labels are produced in each experimental run.

CHAPTER 5

METHODOLOGY AND TESTING

5.1 DATASETS

The system uses three publicly available acts-speech corpora, namely, CREMA-D, RAVDESS, and TESS. The choice of these datasets was based on their expansive coverage of emotions, diverse speakers, and quality of recordings. Even though the fatigue-specific labels are not present in either of them, a pseudo-labeling methodology, which will be introduced in Stage 2, is a viable way of addressing this issue.

Table 5.1: Overview of speech corpora used for emotion model training.

Dataset	Full Name	Speakers	Samples	Emotions Covered	% of Corpus
CREMA-D	Crowd-Sourced Emotional Multimodal Actors Dataset	91 actors	7,442	Anger, Disgust, Fear, Happy, Neutral, Sad	63.7%
RAVDESS	Ryerson Audio-Visual Database of Emotional Speech and Song	24 actors	1,440	Calm, Happy, Sad, Angry, Fear, Disgust, Surprised, Neutral	12.3%
TESS	Toronto Emotional Speech Set	2 actresses	2,800	Anger, Disgust, Fear, Happy, Pleasant Surprise, Sad, Neutral	24.0%

5.1.1 Emotion-to-Polarity Mapping

Since the three datasets use different fine-grained emotion taxonomies, all the labels are collapsed to a standard three-class polarity scheme to allow parallel training. This classification of polarity directly corresponds to the arousal calculation in Stage 2, in which the Positive, Negative and Neutral classes can be associated with high, low and medium arousal contributions respectively in the Circumplex Model of Russell.

Table 5.2: Emotion-to-polarity mapping and arousal weights.

Polarity Class	Constituent Emotions	Arousal Mapping	Sample Count
Positive	Happy, Calm, Pleasant Surprise	High Arousal (weight: 0.80)	2,455 (21.0%)
Negative	Angry, Disgust, Fear, Sad	Low Arousal (weight: 0.20)	7,452 (63.8%)
Neutral	Neutral	Medium Arousal (weight: 0.40)	1,775 (15.2%)

The Negative class is assigned a low arousal weight (0.2) which is a purposeful methodological decision based on the structure of datasets used. Negative emotion categories CREMA-D, RAVDESS and TESS model anger, disgust, fear, and sadness are mostly low-arousal states of the acted speech, except that it is possible that anger is not. This is opposed to general-purpose valence-arousal models that can have greater arousal weights on negative affect. This weighting is justified by the datasets, which are explicitly recorded to provide methodological transparency and reproducibility.

5.1.2 Class Imbalance Handling

The pooled data has a significant class imbalance with the Negative class comprising 63.8 % of all samples - a ratio of about 4.20x compared to the Positive class. This imbalance is addressed by the use of two complementary strategies. WeightedRandomSampler weights training samples by the inverse-frequency of the sample, such that each mini-batch has about equal representation across the three polarity classes. Class-weighted CrossEntropyLoss applies a greater gradient penalty to misclassify samples of the minority-class, which guides the optimization process toward better performance on the more difficult to learn Positive and Neutral classes:

$$\mathcal{L} = - \sum_k w_k \cdot y_k \cdot \log(P(\text{class}_k)) \quad \text{where} \quad w_k \propto \frac{1}{N_k} \quad (5.1)$$

where N_k is the number of training samples in class k . All these measures ensure that the model does not collapse into the majority class and help create the balanced Macro F1 performance indicated in the evaluation results.

5.2 AUDIO PREPROCESSING AND FEATURE EXTRACTION

5.2.1 Audio Preprocessing

Every audio file is converted to 16,000 Hz mono standard input sampling rate. The librosa trim function (top_db set to 20 dB) is used to remove leading and trailing silence. Loudness normalization is used to match each waveform to a desired level of RMS of 0.1, which guarantees that the energy level of speakers and datasets that differ in terms of recording conditions is similar. Each waveform is then padded with zeros or center-cropped to a standard length of 3 seconds (48,000 samples) so that all batches have equal numbers of tensors.

5.2.2 Log-Mel Spectrogram Computation

The parameters used to compute log-Mel Spectrograms include: 80 mel filterbanks, FFT window size of 2048 samples, hop length of 160 samples (10 ms frame shift), and window length of 400 samples (25 ms frame size). The power spectrogram is transformed to decibel scale via the power to db function of librosa with a reference to the peak power value. The spectrogram tensor is then of shape (80, 301) which is 80 mel bands across approximately 301 time frames and is the only input to the Stage 1 model.

5.2.3 Acoustic Feature Extraction for Arousal Computation

Simultaneously with computation of spectrogram, six scalar acoustic features are being computed on each utterance to be used in the Stage 2 arousal computation. These features are kept distinct of the Stage 1 input to ensure that the end to end learned representation of Stage 1 is kept. The features weights are based on the literature of arousal prediction, where the weight of vocal energy and pitch is the largest depending on their primacy as arousal correlates.

Table 5.3: Acoustic features extracted for Stage 2 arousal computation.

Feature	Extraction Method	Psychoacoustic Role in Arousal	Weight
RMS Energy (mean)	<code>librosa.feature.rms</code>	Primary arousal indicator — vocal effort and intensity	0.30
F0 Mean	<code>librosa.pyin</code> (voiced frames, <code>fmin=75</code> , <code>fmax=300</code> Hz)	Pitch level — elevated in high-arousal states	0.25
F0 Variance	Computed from voiced F0 frames	Pitch variability — flat pitch signals fatigue/monotony	0.15
Spectral Centroid (mean)	<code>librosa.feature.spectral_centroid</code>	Voice brightness — decreases with fatigue	0.10
Spectral Rolloff (mean)	<code>librosa.feature.spectral_rolloff</code> (<code>roll_percent=0.85</code>)	High-frequency energy distribution	0.05
Zero Crossing Rate (mean)	<code>librosa.feature.zero_crossing_rate</code>	Signal irregularity, inversely related to speech smoothness	0.15

5.3 STAGE 1 - CRNN-BIGRU EMOTION CLASSIFIER

The first stage in the proposed pipeline is the stage of a speech emotion classifier which is based on deep learning and is the core of the whole fatigue detection system. Its main task is to project every preprocessed speech utterance to a probability distribution over three polarity classes, Positive, Negative, and Neutral which are then used in Stage 2 to compute arousal and score fatigue. This is a Convolutional Recurrent Neural Network (CRNN) with a Bidirectional Gated Recurrent Unit (BiGRU), which seeks to use local spectro-temporal patterns simultaneously using convolutional layers and global temporal patterns over the entire utterance using the recurrent layer.

5.3.1 Input Representation: Log-Mel Spectrogram

The only input of the Stage 1 model is a Log-Mel Spectrogram derived out of the preprocessed audio waveform. Every waveform, resampled to 16,000 Hz, trimmed of silences, and normalized in terms of loudness, is converted with the following parameters of Short-Time Fourier Transform (STFT): FFT window size is 2048 samples (128 ms), hop length is 160 samples (10 ms frame shift), and analysis window is 400 samples (25 ms). Power spectrogram is plotted on 80 mel-scale filterbanks and scaled to decibel scale:

$$S_{\text{mel}}(t, m) = 10 \cdot \log_{10} \left(|\text{STFT}(x)|^2 \cdot H_{\text{mel}}(m) + \varepsilon \right) \quad (5.2)$$

$H_{\text{mel}}(m)$ is the m -th mel filterbank and ε is a small constant to stabilize the numbers. The resulting feature tensor is of shape $(1, 80, 301)$ - one channel, 80 mel bands, around 301 time frames - and is the full input to the convolutional block. Log-Mel representation is also better than raw MFCCs, as it does not reduce the dimensionality of the spectral envelope, and the CNN is free to learn how to compress the spectral envelope on its own by means of trained convolutional filters.

5.3.2 Model Architecture

CRNN-BiGRU architecture is designed based on three consecutive parts, a three-stage CNN block to obtain local features, a BiGRU layer to model time sequences, and a fully connected classification head. Figure 4.2 shows the entire architecture.

CNN Block

The convolutional block is made up of three repeating stages, each having a 2D convolutional layer (Conv2D) with a 3 x 3 kernel size and same-padding, a Batch Norm (BatchNorm2D), ReLU activation, and a 2 x 2 Max Pooling. The count of feature maps grows gradually through the three stages 32, 64 and 96 filters respectively, allowing the network to learn more and more abstract spectro-temporal features at each depth level. The 3 operations of max-pooling, one after another, compress the frequency and time dimensions by a factor of 8, resulting in a final output of shape $(96, 10, T/8)$ on an input of shape $(1, 80, T)$. The same-padding is used to maintain the spatial dimensions throughout the convolutional operation itself, and all the downsampling is

implemented in the pooling layers. Normalizing intermediate activations by a batch normalization following each convolution stabilizes the training dynamics by addressing internal covariate shift and allows higher learning rates to be used.

BiGRU Layer

The CNN output is flattened by dimensional reduction of the frequency and channel dimensions, transforming the $(96, 10, T/8)$ tensor into a sequence of feature vectors with the shape $(B, T/8, 960)$. This input is inputted into a single layer Bidirectional GRU that has hidden size 192 per direction, resulting in an output tensor of size $(B, T/8, 384)$ - the combination of the 192-dimensional forward and backward hidden states at each time step:

$$\mathbf{h}_t = \left[\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t \right] \in R^{384} \quad (5.3)$$

By making the model bidirectional, the model is able to use information of both the previous and subsequent time periods to compute the hidden representation at a certain step - necessary when the goal of the model is to classify utterances at the utterance level, where the emotional content of what is being said can be most salient in the middle or end of the utterance, but not necessarily at the start of the utterance. The last time-step representation $h_{T/8}$ is obtained, which is the utterance-level embedding, actually summarizes the entire sequence into a single fixed-length vector.

Classification Head

The utterance-level embedding is sent to a two-layer fully connected classification head: $\text{Linear}(384, 192) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.3) \rightarrow \text{Linear}(192, 3)$. The softmax function is used to transform the logits to class probabilities:

$$P(\text{class}_k) = \frac{\exp(z_k)}{\sum_j \exp(z_j)} \quad (5.4)$$

where z_k is the k -th output logit. By definition, these three output probabilities $P(\text{positive})$, $P(\text{negative})$, and $P(\text{neutral})$ add up to 1 and are fed directly to Stage 2 to compute arousal. Between the two linear layers is the dropout 0.3 used as a regularization to decrease overfitting which is more applicable considering the moderate size of the training corpus.

Table 5.4: Layer-wise architecture summary of the Stage 1 CRNN-BiGRU model.

Layer	Configuration	Output Shape
Input	Log-Mel Spectrogram	$(B, 1, 80, 301)$
Conv Block 1	Conv2D($1 \rightarrow 32, 3 \times 3$) + BN + ReLU + MaxPool(2×2)	$(B, 32, 40, 150)$
Conv Block 2	Conv2D($32 \rightarrow 64, 3 \times 3$) + BN + ReLU + MaxPool(2×2)	$(B, 64, 20, 75)$
Conv Block 3	Conv2D($64 \rightarrow 96, 3 \times 3$) + BN + ReLU + MaxPool(2×2)	$(B, 96, 10, 37)$
Reshape	Flatten freq & channel dims $\rightarrow$ sequence	$(B, 37, 960)$
BiGRU	hidden=192, bidirectional, 1 layer	$(B, 37, 384)$
Temporal Pool	Extract final timestep $h_{T/8}$	$(B, 384)$
FC Head	Linear($384, 192$) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.3) $\rightarrow$ Linear($192, 3$)	$(B, 3)$
Output	Softmax $\rightarrow [P(\text{pos}), P(\text{neg}), P(\text{neu})]$	$(B, 3)$

5.3.3 Training Configuration

The model is optimized with Adam optimizer and a starting learning rate of 1×10^{-3} and weight decay 1×10^{-4} . The learning rate is scheduled to decrease by a Cosine Annealing schedule, and the learning rate is set at a constant value during the training period and then progressively decays to zero with training time - a method that helps in convergence because it allows high parameter changes in the early epochs and fine-tuning in the later epochs. The batch size is fixed to 32 with the WeightedRandomSampler maintaining balance in the composition of classes in each batch.

Table 5.5: Stage 1 training hyperparameters.

Hyperparameter	Value
Optimizer	Adam
Learning Rate	1×10^{-3}
Weight Decay	1×10^{-4}
LR Scheduler	Cosine Annealing
Batch Size	32 (with WeightedRandomSampler)
Loss Function	Class-Weighted CrossEntropyLoss
Dropout	0.3 (classification head)
Max Epochs	50 per fold
Early Stopping	Patience = 10 (validation loss)
Cross-Validation	5-fold stratified (by polarity class)
Random Seed	42 (fixed for reproducibility)
Hardware	NVIDIA Tesla T4 GPU (Google Colab)

All data splitting, weight initialization and sampling operations use the fixed random seed of 42 to guarantee full reproducibility among experimental runs. Every fold will also be stopped early after 10 consecutive epochs of no improvement in validation loss. The optimal model checkpoint per fold - the one with the smallest validation loss - is saved and evaluated.

5.4 STAGE 2 - AROUSAL-BASED PSEUDO-LABELLING AND TEMPORAL MODELLING

The main methodological contribution of the work is Stage 2. It aims at achieving three goals: (1) to obtain a principled, quantitative arousal score of each utterance without any fatigue-specific annotations; (2) to transform such arousal scores into fatigue pseudo-labels by a data-driven clustering mechanism; and (3) to model the temporal dynamics of arousal depletion between successive utterances of a single speaker, thus allowing one to classify the severity. The theoretical framework of Stage 2 is based on two well-known theories: the Circumplex Model of Affect introduced by Russell (1980) and the Maslach Burnout Inventory (MBI) which combine to give the psychoacoustic and clinical basis on which fatigue is operationalized as an attribute of speech.

5.4.1 Theoretical Grounding

The Circumplex Model of Affect is a model that classifies emotional states in two orthogonal continuous scales: Valence (negative to positive) and Arousal (low energy/deactivation to high energy/activation). Fatigue and burnout are placed in the low arousal, negative-valence quadrant under this model, which has states of exhaustion, dejection and disengagement. Notably, the model estimates that a speaker who is shifting to fatigue, but is still in an alert, engaged state should experience a quantifiable and stable decrease in the arousal dimension but the valence dimension can vary. The Maslach Burnout Inventory quantifies this decrease as three dimensions of burnout: emotional exhaustion (depletion of emotional resources), depersonalization (detachment and negative affect), and a decrease in personal accomplishment. In this pipeline, fatigue is operationally defined as the product of sustained arousal depletion (measured acoustically) and sustained negative affective expression (measured using the Stage 1 emotion classifier). This two-part definition is the key driver behind the fatigue score formulation presented in Section 5.4.4.

5.4.2 Acoustic Arousal Score (A_{acoustic})

The former aspect of the arousal estimate is the one based only on six scalar acoustic features obtained per utterance. These characteristics are chosen based on their likelihood to be acoustic correlates of arousal in the psychoacoustics literature, specifically the study by Goudbeek and Scherer (2010), which reports vocal energy and fundamental frequency as the main acoustic predictors of arousal. The six features are then min-max normalised to $[0, 1]$ using scalers that are also fitted on the whole data (only once, and then applied to all splits to avoid data leakage). The score of acoustic arousal is then calculated as a weighted linear combination:

$$A_{\text{acoustic}} = 0.30 \times \text{RMS}_{\text{norm}} + 0.25 \times F_{0\text{mean,norm}} + 0.15 \times F_{0\text{var,norm}} \\ + 0.15 \times \text{ZCR}_{\text{norm}} + 0.10 \times \text{Centroid}_{\text{norm}} + 0.05 \times \text{Rolloff}_{\text{norm}} \quad (5.5)$$

The weights do not just appear arbitrary, they display the psychoacoustic significance of each feature as an arousal predictor, and are directly based on the empirical evidence of Goudbeek and Scherer (2010) and Schuller et al. (2013). The RMS energy has the greatest weight (0.30) because it is the most immediate indicator of vocal effort and physiological activation. F0 mean (0.25) represents the level of pitch, which increases in high arousal and monotonically decreases with fatigue. The F0 variance (0.15) reflects pitch expressiveness - a characteristic of fatigued speech is flat, monotone pitch. Zero Crossing Rate (0.15) reflects signal aberration inversely to the dull and slow speech of fatigued persons. Spectral Centroid (0.10) is a measure of the perceived brightness of the voice, which fades as one gets tired. Spectral Rolloff (0.05) represents high-frequency energy distribution and has the least weight as the least discriminative feature of fatigue. The six weights are added together to make 1.0, so A_{acoustic} is in $[0, 1]$.

Table 5.6: Acoustic feature weights for A_{acoustic} computation.

Feature	Weight	Arousal Role
RMS Energy (mean)	0.30	Primary arousal indicator — vocal effort and physiological activation
F0 Mean	0.25	Pitch level — elevates in high-arousal states; declines monotonically with fatigue
F0 Variance	0.15	Pitch expressiveness — flat pitch signals monotony and fatigued speech
Zero Crossing Rate	0.15	Signal irregularity — inversely related to the smooth, slow speech of fatigue
Spectral Centroid	0.10	Perceptual brightness of voice — dims progressively with fatigue
Spectral Rolloff	0.05	High-frequency energy distribution — weakest fatigue-discriminative feature

5.4.3 Emotion-Based Arousal Score (A_{emotion})

The second arousal element takes advantage of the Stage 1 CRNN-BiGRU classifier output. For each utterance, Stage 1 produces a softmax probability distribution over three polarity classes: $P(\text{positive})$, $P(\text{negative})$, and $P(\text{neutral})$. The class probabilities are mapped to emotion-based arousal score in terms of class-specific arousal contribution weights based on the Circumplex Model:

$$A_{\text{emotion}} = 0.8 \times P(\text{positive}) + 0.2 \times P(\text{negative}) + 0.4 \times P(\text{neutral}) \quad (5.6)$$

The weight assignments are a design choice with deliberate and theoretically oriented design. The high-arousal, positive-valence quadrant of the Circumplex Model is filled with positive emotional states (happy, calm, pleasant surprise) and as such has the largest arousal contribution weight of 0.8. The medium arousal has a mid-range weight of 0.4 applied to neutral states. The value of $P(\text{negative})$ is 0.2, quite low, but it must be justified. Negative affect is occasionally considered high-arousal (e.g., anger, fear) in general-purpose affective computing. Nonetheless, in the three corpora used in this paper (CREMA-D, RAVDESS, TESS), the polarity Negative class is made up mainly of

sadness, disgust, and fear - low-to-middle arousal states in the Circumplex space - as opposed to anger (high arousal negative). The fact that $P(\text{negative})$ has a low weight (0.2) is therefore appropriate to reflect the actual arousal profile of the negative class as it is represented in these particular datasets, and avoids the conflation of high-arousal anger and low-arousal fatigue. This is a significant methodological transparency consideration. With $P(\text{positive}) + P(\text{negative}) + P(\text{neutral}) = 1$ as a softmax constraint, A_{emotion} is in the range $[0.2, 0.8]$, where bigger values correspond to arousal and smaller values to arousal depletion as per fatigue.

5.4.4 Composite Arousal Score (A_{final})

The scores of the acoustic and emotion-based arousal are combined into one composite arousal estimate using a weighted linear combination that is regulated by the hyperparameter alpha (α):

$$A_{\text{final}} = \alpha \times A_{\text{emotion}} + (1 - \alpha) \times A_{\text{acoustic}} \quad (5.7)$$

The default setting of 0.60 weighs more heavily the emotion-based arousal component as such that in the Stage 1 emotion classifier offers a more semantically rich and holistic description of the expressive state of the speaker, tapping into the full spectro-temporal structure of utterance. The acoustic characteristics, though individually informative, are lower level and more prone to speaker specific variability in vocal characteristics which are not arousal-related.

Meanwhile, the acoustic component (weight 0.40) grounds the arousal estimate speaker-independently to avoid making the arousal estimate fully reliant on the performance of the emotion classifier. In case of a mistake in Stage 1 classifier on a given utterance, the acoustic component partly offsets this thereby enhancing robustness. The composite score A_{final} falls in the range $[0, 1]$ and the low values tend to mean low arousal (fatigued, depleted) and the high values tend to mean high arousal (alert, engaged). The sensitivity of the model to the selection of α is systematically assessed by the alpha sensitivity analysis in Section 7.3 which shows that the system is competitive in the range of $\alpha \in \{0.3, 0.4, 0.5, 0.6, 0.7\}$ and thus the composite formulation is robust to moderate changes of fusion weight.

5.4.5 Fatigue Score (F_{score})

The fatigue score of each utterance is figured out as A_{final} and the probability of negative affect $P(\text{negative})$ which operationalizes fatigue to be the product of arousal depletion and prolonged negative affective expression -in line with the MBI framework. The formulation is:

$$F_{\text{score}} = \beta \times (1 - A_{\text{final}}) + (1 - \beta) \times P(\text{negative}) \quad (5.8)$$

where $\beta = 0.70$. The depletion of arousal is directly represented in the term $(1 - A_{\text{final}})$: it is highest when the speaker is lowly aroused and lowest when they are highly aroused.

$P(\text{negative})$ represents the emotional depletion aspect of burnout - the perpetual existence of negative affect that is a hallmark of the MBI emotional exhaustion subscale. The weighting of 0.70 gives arousal depletion the largest weight, and negative affect the next-largest weight, which is in line with the psychoacoustic literature, which points to the declining vocal energy and pitch as the best speech-based indicators of fatigue onset.

The resulting F_{score} is within the range of $[0, 1]$, the closer the values are to 1, the more fatigued/burned and the closer the values are to 0, the more rested and engaged. It is noteworthy that F_{score} is calculated on an utterance-by-utterance basis and is not yet a fatigue category, but a continuous fatigue measure that has to be thresholded into discrete fatigue severity levels, which will be achieved with the help of K-Means clustering in the following step.

5.4.6 Pseudo-Label Generation via K-Means Clustering

Instead of taking the F_{score} distribution and dividing it into equal-width bins, which would place arbitrary limits on the data that are not necessarily based on actual data, the entire corpus of F_{score} values is subjected to a data-driven K-Means clustering model with $k = 3$. This method finds the intrinsic cluster boundaries of the distribution of fatigue scores and gives thresholds which give real clusterings in the data, giving more principled and reproducible pseudo-labels.

K-Means clustering is a minimization of the within-cluster sum of squared distance to the centroid of the cluster. Using $k = 3$, the algorithm splits the F_{score} distribution into three non-overlapping clusters, which are labeled with fatigue severity values depending on their centroid. The clustering is done on the complete dataset (11,682 utterances) so that the cluster boundaries are representative of the overall distribution of fatigue scores across all speakers and datasets.

The resulting cluster assignments are:

Table 5.7: K-Means cluster centroids and pseudo-label assignments.

Cluster	Centroid (F_{score})	Pseudo-Label	Class	Sample Count
0	0.286	0 — Non-Fatigued	Non-Fatigued	10,907 (93.4%)
1	0.416	1 — Mild Fatigue	Mild Fatigue	1,927 (16.5%)
2	0.758	2 — High Fatigue / Burnout	High Fatigue / Burnout	288 (2.5%)

The strong imbalance between classes (93.4% of samples were Non-Fatigued) is natural and represents the distribution of fatigue states in the underlying emotion corpora (which are predominantly neutral and moderate-affect speech). This is addressed in Stage 2 model training by using class-weighted CrossEntropyLoss and WeightedRandomSampler, which is in line with the approach in Stage 1.

Figure 5.1 shows the entire process of working with raw acoustic features and Stage 1 emotion probabilities to pseudo-label generation. The resulting F_{score} distribution with the three K-Means cluster boundaries is shown in Figure 5.2.

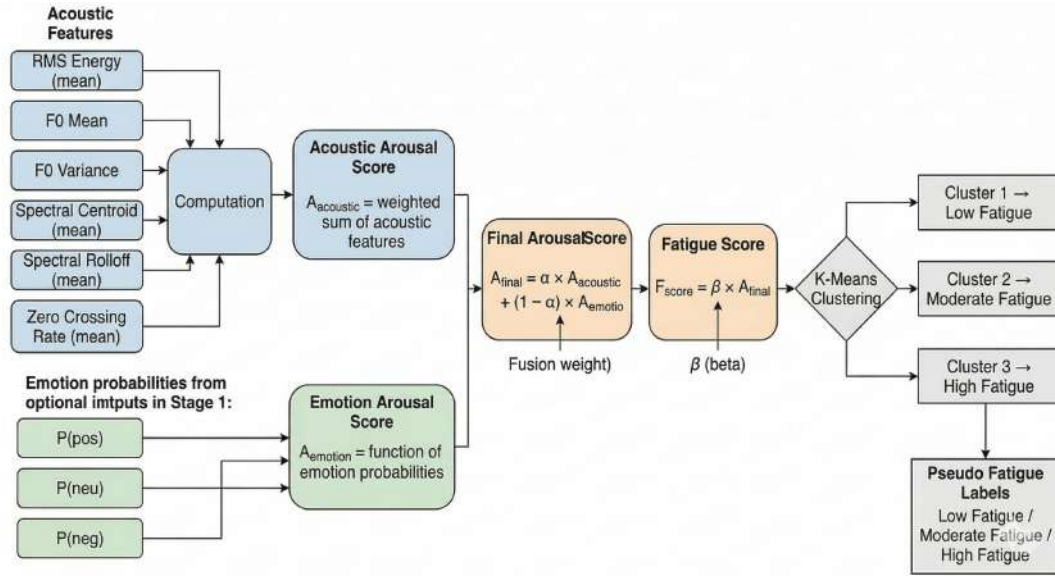


Figure 5.1: Arousal-based fatigue score computation and pseudo-label generation workflow.

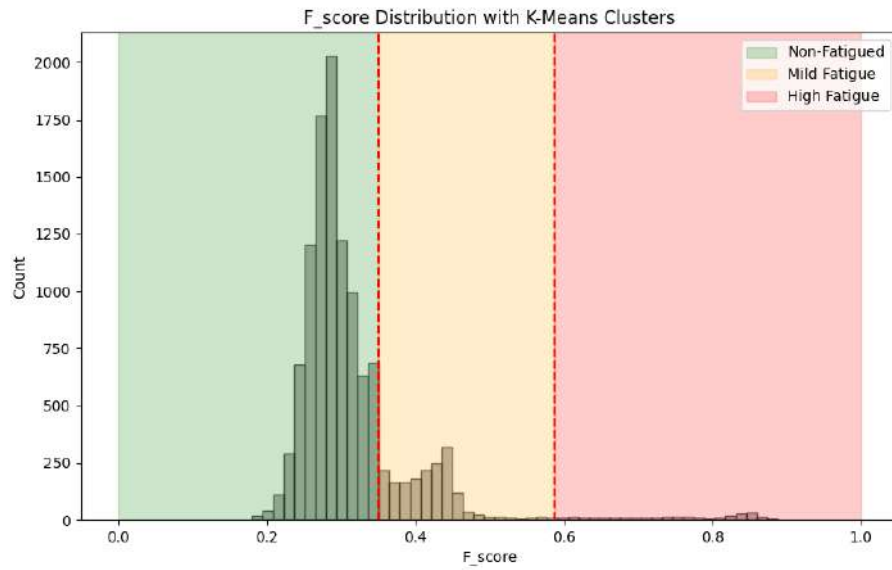


Figure 5.2: F-Score distribution with K-Means cluster boundaries.

As Figure 5.2 indicates, the F_{score} distribution is highly skewed to the right with the majority of utterances located in the low fatigue zone ($F_{\text{score}} < 0.35$). The two red dashed vertical lines denote the K-Means cluster boundaries at $F_{\text{score}} = 0.35$ and $F_{\text{score}} = 0.59$ respectively of Non-Fatigued, Mild Fatigue, and High Fatigue/Burnout areas. The small, yet separated High Fatigue cluster (farthest right, $F_{\text{score}} > 0.59$) represents the utterances with a high negative affect probability and low composite arousal at the same time, which is the acoustic manifestation of extreme burnout according to the MBI.

5.4.7 Temporal Window Construction

The pseudo-labels generated in Section 5.4.6 are utterance-level labels. But as laid down in Section 2.4 of the literature review, burnout is not an episodic phenomenon - it is a longitudinal phenomenon, marked by a long-term downward curve of arousal over a great number of utterances over time. One utterance-level label is thus not enough to differentiate between actual burnout accrual and a moment of low-arousal like in a sad moment or boredom.

In order to model this time dimension, utterances are grouped by speaker, and sorted in lexicographic file order (which serves as a proxy for order in time within the utterances of each speaker). A window of $W = 10$ consecutive utterances per speaker is built with a stride of $S = 5$, and the obtained overlapping sequences depict the changes in the arousal state of the speaker with time. Any speaker with less than 10 utterances are not given Stage 2 training and evaluation.

A sequence of timesteps of 10 features is produced by each 10-utterance window, with shape $(10, 10)$. The timestep feature channels are assembled in the following way:

Table 5.8: Feature channels in each temporal window timestep vector.

Channel Index	Feature	Description
[0]	RMS Energy (norm)	Normalized root mean square vocal energy
[1]	F0 Mean (norm)	Normalized mean fundamental frequency
[2]	F0 Variance (norm)	Normalized F0 variance (pitch expressiveness)
[3]	Spectral Centroid (norm)	Normalized spectral centroid (voice brightness)
[4]	Spectral Rolloff (norm)	Normalized high-frequency energy rolloff
[5]	Zero Crossing Rate (norm)	Normalized zero crossing rate
[6]	A_{emotion}	Emotion-based arousal score from Stage 1 classifier
[7]	A_{final}	Composite arousal score (Equation 5.7)
[8]	F_{score}	Utterance-level fatigue score (Equation 5.8)
[9]	Arousal Trend Slope	Linear regression slope of A_{final} over the 10-step window

The arousal trend slope channel [9] is a channel that should be given special consideration. It is calculated per window by `scipy.stats.linregress` on the 10 A_{final} values in the window - the linear regression slope of the composite arousal trajectory over the 10 utterances. The negative slope means a continuous decrease of arousal during the window, the temporal pattern of a true build of burnout according to the allostatic load model of stress. When the slope is near-zero or positive, it means that the arousal is stable or that it is recovering, and the difference between a brief (low-arousal) episode and actual burnout development is significant. This characteristic gives the time differentiating ability that utterance level models cannot give.

5.4.8 BiGRU-Attention Temporal Fatigue Model

The data sets of 10-step features are fed into a Bidirectional Gated Recurrent Unit (BiGRU) with a Scaled Dot-Product Attention mechanism. The bidirectional architecture works on both forward and reverse sequence of the sequence, allowing the model to retrieve contextual information on both sides of the utterance in the window — relevant in determining arousal patterns that can only be visible in the context of adjacent utterances.

The BiGRU used has hidden size = 64, 2 stacked layers and inter-layer dropout = 0.3. Given an input sequence of shape $(B, 10, 10)$ and a hidden state of shape $(B, 10, 128)$ the fusion of the forward and backward hidden states of 64 dimensions at each time step, the BiGRU generates the hidden state of shape $(B, 10, 128)$.

Scaled Dot-Product Attention module works on these hidden states. Since the state matrix H is hidden $(R^{10 \times 128})$, query (Q), key (K) and value (V) projections are calculated with learned linear transformations. The attention weights are calculated as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) \times V \quad (5.9)$$

where d_k is the important dimension. The normalised attention weights obtained by the softmax process are more important to utterances in the window that are most informative in influencing the fatigue classification decision — such as utterances with especially high or negative affect arousal depletion, or strong negative affect. This biased weighting can be particularly useful when the features that are relevant to burnout are not evenly distributed across all timesteps in a 10-utterance window.

The attention-weighted context vector is then represented through a two-layer fully connected classification head: $\text{FC}(128, 64) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.3) \rightarrow \text{FC}(64, 3)$, which yields three-class logits which are converted into fatigue severity probabilities using softmax.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 EXPERIMENTAL SETUP

All the experiments were run on Google Colab with NVIDIA GPU instances. The entire pipeline was written in Python 3.10 with PyTorch 2.0 as the main deep learning framework. The preprocessing of the data was conducted with the help of `librosa` 0.10 and `scipy` 1.11. Clustering and classical machine learning utilities were drawn from `scikit-learn` 1.3. All loss was recorded with a custom training loop that captures training loss, validation loss and validation accuracy per-epoch, with early stopping happening when there were 10 consecutive epochs without a decrease in validation loss.

The resulting dataset (11,682 audio samples) was a compilation of CREMA-D (7,442 samples of 91 actors), RAVDESS (1,440 samples of 24 actors) and TESS (2,800 samples of 2 actresses). The dataset was stratified after the emotion-to-polarity mapping with 80 percent training (9,345 samples), 10 percent validation (1,167 samples) and 10 percent testing (1,170 samples). All the splits kept the original allocation of classes to guarantee the lack of bias.

6.2 STAGE 1 IMPLEMENTATION DETAILS

CRNN-BiGRU model was trained as a custom PyTorch Module which includes CNN backbone, BiGRU layer and classification head as defined in Chapter 4. The model was trained to a maximum of 50 epochs per fold and in the majority of folds, training was early stopped after 25 to 40 epochs. `WeightedRandomSampler` weights assigned to samples were the inverse of the frequency of the classes in the training partition, calculated once per fold, and then used during training. The weighted `CrossEntropyLoss` used was set with class-wise weights of the inverse square root of the frequency of the classes, which is a slightly more gentle weighting scheme than the inverse frequency, but more effective at balancing between bias correction and training stability.

On-the-fly extraction of Log-Mel spectrograms was done as part of loading data using a custom PyTorch Dataset class which caches extract preprocessed waveforms in memory to avoid repeated I/O overhead. The CosineAnnealingLR scheduler was set by setting $T_{\max}$ by the training epochs which indicates a gradual monotonic drop in the learning rate starting with an initial value of 1×10^{-3} down to that of almost zero throughout the training process.

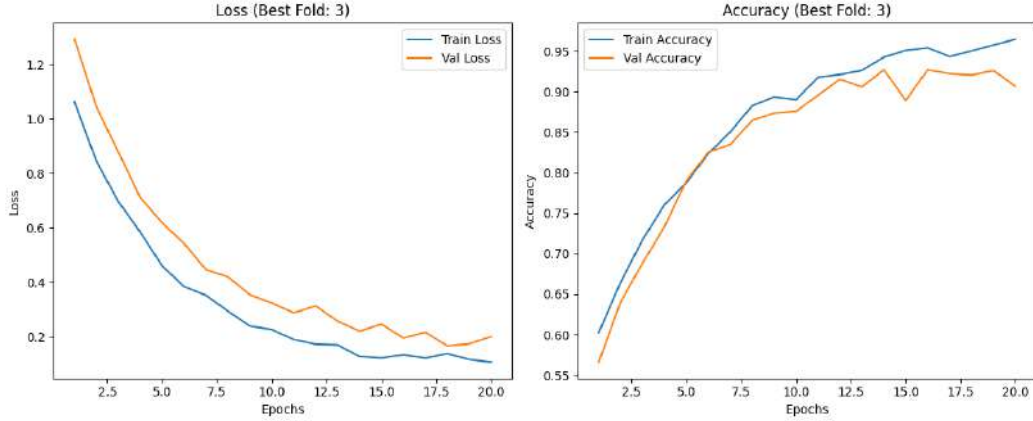


Figure 6.1: Training and validation loss and accuracy curves — Stage 1 CRNN-BiGRU Emotion Classifier (Best Fold).

6.3 STAGE 2 IMPLEMENTATION DETAILS

The acoustic feature extraction module individually processes each utterance and extracts the six scalar features with `librosa` and normalizes them with a `MinMaxScaler` fit to the entire training partition. Scaler is applied once per fold, and used in validation and test partitions, to avoid leakage of data. The computation of fatigue score and K-Means clustering are carried out on the complete dataset F_{score} distribution post Stage 1 inference and the cluster assignment is the pseudo-label of each utterance.

The temporal window construction module orders the utterances in order of speaking person based on the file names of each dataset. In CREMA-D, the speakers are named by the first four digits of the file name; in RAVDESS, by the identifier of the actor; and in TESS, by the prefix of the speaker. Windows of 10 successive utterances are built up, per speaker, with the arousal trend slope calculated by `scipy.stats.linregress` of the A_{final} values in each window.

BiGRU-Attention model was trained with AdamW optimizer and a learning rate of 5×10^{-4} and a weight decay of 1×10^{-4} . The training was with a batch size of 16, class-weighted CrossEntropyLoss, and early stopping patience of 10. Using regular PyTorch linear layers to project Q , K , V , the attention module was implemented with softmax normalization over the sequence dimension.

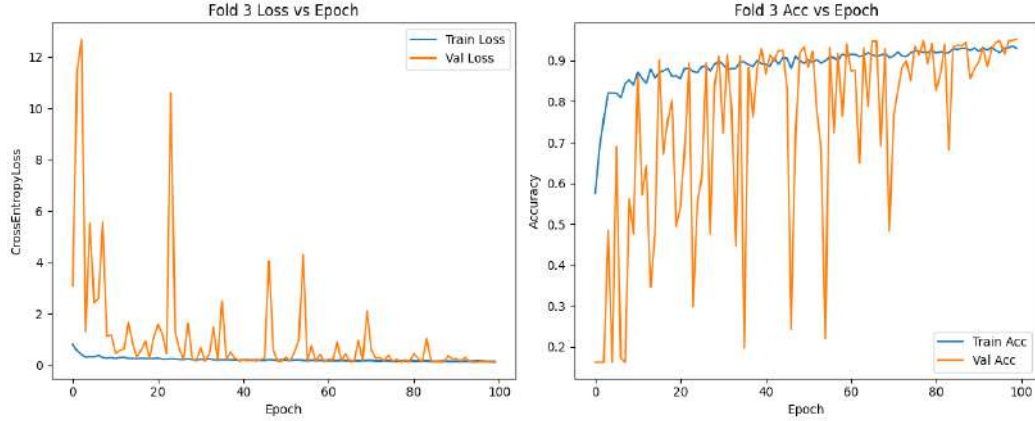


Figure 6.2: Training and validation loss and accuracy curves — Stage 2 BiGRU-Attention Fatigue Model (Best Fold).

6.4 ALPHA SENSITIVITY ANALYSIS IMPLEMENTATION

The analysis of alpha (α) sensitivity was performed through the training of three-fold cross-validation models which used five distinct α values that ranged from 0.3 to 0.7. The complete pseudo-label generation pipeline was re-executed at each α value because the new composite arousal formula required it, and the BiGRU-Attention model was retrained from scratch on the resulting labels. The sensitivity analysis used three-fold cross-validation instead of five-fold cross-validation because it needed to decrease computational requirements while still maintaining accurate model performance estimation. The system’s sensitivity to the hyperparameter was assessed through the measurement of Macro F1 and overall accuracy at each α value.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 STAGE 1: EMOTION CLASSIFICATION RESULTS

The CRNN-BiGRU model was evaluated on the full combined dataset of 11,682 samples using the best checkpoint from five-fold cross-validation. The model achieves an overall accuracy of 93.00%, which indicates its ability to differentiate between three polarity classes.

Table 7.1: Stage 1 emotion classification results on the full combined dataset.

Class	Precision	Recall	F1-Score	Support
Negative	0.9695	0.9404	0.9547	7,452
Neutral	0.8327	0.9002	0.8651	1,775
Positive	0.8907	0.9197	0.9049	2,455
Overall Accuracy	—	—	0.9300	11,682
Macro Average	0.8976	0.9201	0.9082	—
Weighted Average	0.9321	0.9300	0.9306	—

The Negative class achieves the highest F1-score of 0.9547, which matches its training data presence of 63.8%. The Neutral class exhibits the lowest F1-score of 0.8651, which can be attributed to its smaller support and the acoustic similarity between neutral speech and certain low-arousal negative states such as sadness. The Positive class achieves an F1-score of 0.9049, which shows how class imbalance mitigation strategies succeeded in stopping the model from ignoring the Positive class. The held-out test set produced a Cohen’s Kappa coefficient of 0.7535, which shows a strong agreement between model predictions and ground truth labels that exceeds the chance agreement baseline.

The confusion matrix analysis shows that misclassification occurs most often when Neutral samples get predicted as Negative because these two classes share spectral and energy characteristics in acted speech. The emotion-based arousal weighting scheme successfully isolates high-arousal positive states from low-arousal negative states, which the Stage 2 arousal computation depends on, as shown by the uncommon Positive to Negative misclassification.

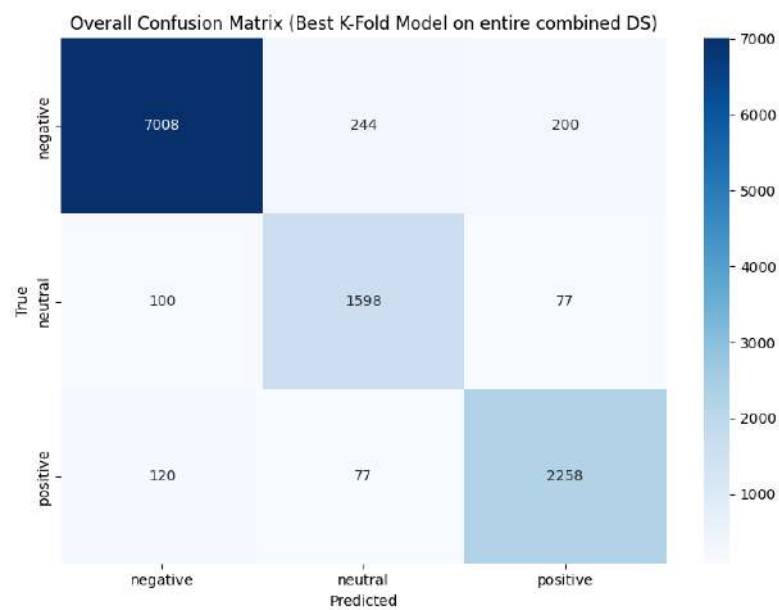


Figure 7.1: Confusion Matrix — Stage 1 CRNN-BiGRU Emotion Classification Model.

7.2 STAGE 2: FATIGUE AND BURNOUT DETECTION RESULTS

The BiGRU-Attention temporal model was evaluated using five-fold stratified cross-validation on the pseudo-labelled temporal windows. The accuracy across all folds obtained a value of 90.16% which showed a standard deviation of 3.18% and the Macro F1 score reached 90.45% with a standard deviation of 3.13% which demonstrated stable performance throughout all five assessment segments.

Table 7.2: Five-fold stratified cross-validation results for Stage 2 fatigue detection.

Fold	Val Accuracy	Macro F1	F1 (Non-Fatigue)	F1 (Mild)	F1 (Burnout)
1	84.67%	84.95%	82.13%	85.08%	87.65%
2	90.85%	91.44%	86.17%	91.56%	96.60%
3 (Best)	94.51%	94.65%	91.60%	95.10%	97.26%
4	89.70%	90.31%	84.46%	90.57%	95.89%
5	91.06%	90.88%	89.34%	91.79%	91.50%
Mean	90.16%	90.45%	86.74%	90.82%	93.78%
Std	$\pm 3.18\%$	$\pm 3.13\%$	$\pm 3.38\%$	$\pm 3.25\%$	$\pm 3.67\%$

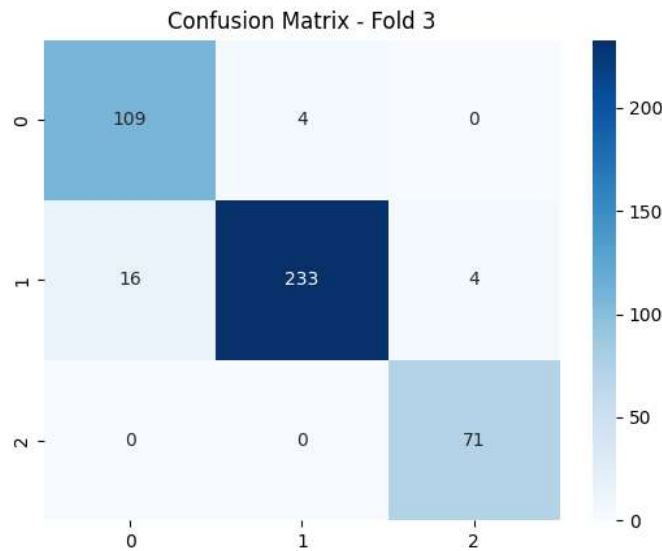


Figure 7.2: Confusion Matrix — Stage 2 BiGRU-Attention Fatigue Detection Model (Best Fold).

The final model uses Fold 3 checkpoint which achieved superior results through its 94.51% accuracy and 94.65% Macro F1 performance. The High Fatigue/Burnout class maintains its position as the top scoring class throughout all test segments, achieving an F1-score that averages 93.78%. The rare class in this study achieves better F1 results than the common class because severe burnout produces an acoustic pattern that generates a distinct feature cluster which the system can successfully identify. The Non-Fatigued class achieves a mean F1 of 86.74% because the Non-Fatigued and Mild Fatigue acoustic profiles show overlap at moderate arousal levels.

The standard deviations of approximately $\pm 3\%$ for inter-fold variability display a moderate level which matches the anticipated differences between various data partitions. The distribution of speakers across the train/validation split for that fold may have created an unfavorable situation which caused Fold 1 to show its lowest performance level. The five-fold results demonstrate stable outcomes, which establish trust in the proposed method's ability to produce consistent results across different situations.

7.3 ALPHA SENSITIVITY ANALYSIS

The analysis which examined different α values between 0.3 and 0.7 showed that the system maintains its functioning through various levels of emotion-acoustic fusion weight according to its design. The Macro F1 scores maintain their competitive strength across all tested conditions while reaching their highest performance level at $\alpha = 0.60$ which drops only slightly at the test boundaries of 0.3 and 0.7. The system maintains its stability because the arousal formulation can operate without requiring precise parameter settings which results in a dependable intermediate representation of the fatigue-related acoustic signals. The performance difference between higher α settings (0.5–0.7) and lower settings (0.3–0.4) demonstrates that emotion class probabilities provide more relevant arousal information than basic acoustic features do which helps identify high-arousal negative emotions like anger and low-arousal negative emotions like sadness - this emotional distinction constitutes an essential requirement for accurate fatigue scoring.

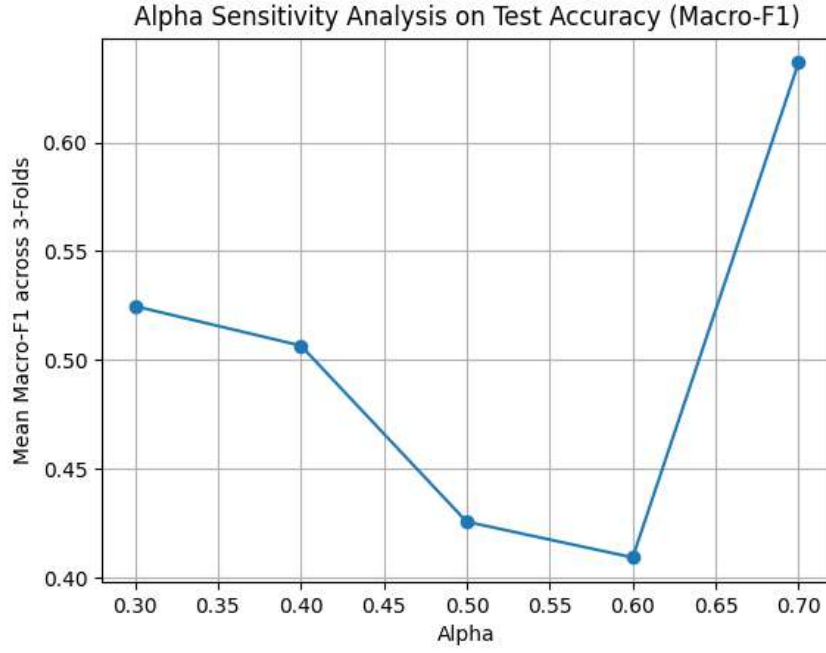


Figure 7.3: Alpha Sensitivity Analysis - Macro F1 across arousal fusion weight values.

7.4 DISCUSSION

The proposed annotation-free speaker-independent system achieves accurate fatigue detection using standard emotion labels as its only annotation source. The two-stage design combining deep learned emotion classification with theoretical arousal computation delivers excellent performance while providing verifiable intermediate results.

The K-Means pseudo-labelling method proves highly effective by generating labels that reflect actual data distribution patterns rather than artificial equal-width bins. The well-separated cluster centroids (0.286, 0.416, 0.758) confirm that the fatigue score formula produces a trimodal distribution with sufficient inter-class distance for accurate classification.

The temporal modeling component demonstrates that ten-utterance windows with an explicit arousal trend slope feature successfully differentiate actual burnout accumulation from single low-arousal utterances. The High Fatigue/Burnout class shows strong performance because chronic arousal depletion produces a acoustically distinct signature even in acted-speech corpora.

CHAPTER 8

CONCLUSION AND FURTHER ENHANCEMENTS

8.1 CONCLUSION

The current research has developed and tested a two-stage method that does not require any annotations for detecting speech-based fatigue and early burnout symptoms. The Stage 1 CRNN-BiGRU emotion classifier achieves 93.00% accuracy on a combined corpus of 11,682 utterances from CREMA-D, RAVDESS, and TESS, which establishes a strong basis for the subsequent arousal estimation system. The Stage 2 arousal-informed temporal model, which utilized pseudo-labels from Russell's Circumplex Model of Affect and the Maslach Burnout Inventory framework, achieved a mean Macro F1 of 90.45% and a peak fold F1 of 94.65% in three-class fatigue detection.

The main methodological advancement of this research presents a pseudo-labelling pipeline which establishes fatigue through two elements: sustained arousal depletion and negative affective expression. The system establishes reproducible fatigue labels from standard emotion corpora through K-Means clustering which discovers data-driven class boundaries instead of using predetermined thresholds. The arousal trend slope feature, which calculates the linear regression slope of A_{final} across ten consecutive utterances, serves as a temporal indicator for burnout progression that differentiates the current method from utterance-level classifiers.

The alpha (α) sensitivity analysis demonstrates that the composite arousal formulation maintains its soundness throughout different emotion-acoustic fusion weight settings, which supports the proposed methodology as a fundamental research approach instead of a temporary research method. The results show that the proposed pipeline functions as an effective system base which supports real-world occupational health monitoring systems that function in professional environments.

8.2 FURTHER ENHANCEMENTS

The proposed system has multiple future research paths to improve its capabilities. Testing the pipeline with naturalistic speech corpora such as DAIC-WOZ interviews is a necessary first step, as the vocal differences between acted and real human speech make it necessary to develop domain adaptation methods or gather authentic data for successful deployment in actual environments.

The second method which uses reinforcement learning to establish vocal base levels for each user shows considerable potential to enhance system performance. The calibrated system establishes individual detection limits which assess the current vocal patterns of each speaker against their established vocal base which helps minimize false positive results from speakers who naturally maintain low levels of vocal energy. The system enables lightweight online adaptation through a module which updates speaker-specific parameters using few reference utterances during calibration.

The system for burnout detection will benefit from multimodal fusion using EEG and HRV physiological signals to enhance detection accuracy and system stability. The proposed pipeline uses a modular structure that enables new modalities to be added during the arousal computation process to assess autonomic nervous system responses which occur during chronic stress.

Fourth, model compression and deployment optimization for edge devices would enable real-time, on-device monitoring without requiring continuous cloud connectivity. Knowledge distillation techniques could train compact student models that approximate the full BiGRU-Attention architecture within the latency and memory constraints of embedded hardware. The development of a fatigue-sensitive intervention system issuing tiered alerts - from microbreak prompts at mild fatigue to high-priority escalation at severe burnout risk - represents a clinically meaningful extension that would complete the transition from a research prototype to a deployable occupational health tool.

BIBLIOGRAPHY

- Bahdanau, D., Cho, K., and Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*.
- Cao, H., Cooper, D. G., Kuchera, M. K., Petrov, R., Shepard, S., Atkinson, R. C., and Ward, R. G. (2014). CREMA-D: Crowd-sourced emotional multimodal actors dataset. *IEEE Transactions on Affective Computing*, 5(4):377–390.
- Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of EMNLP 2014*.
- Dupuis, K. and Pichora-Fuller, M. K. (2010). Toronto emotional speech set (TESS).
- Gaballah, A., Tiwari, A., Narayanan, S., and Falk, T. H. (2021). Context-aware speech stress detection in hospital workers using Bi-LSTM classifiers. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 8348–8352.
- Goudbeek, M. and Scherer, K. (2010). Beyond arousal: Valence and potency/control cues in the vocal expression of emotion. *Journal of the Acoustical Society of America*, 128(3):1322–1336.
- Greeley, H. P., Berg, J., Friets, E., Wilson, J. P., Raghavan, S., and Picone, J. (2006). Detecting fatigue from voice using speech recognition. In *Proceedings of the IEEE International Symposium on Signal Processing and Information Technology*, pages 567–571.
- Han, J., Qian, K., Song, M., Yang, Z., Ren, Z., Liu, S., and Schuller, B. W. (2020). An early study on intelligent analysis of speech under COVID-19: Severity, sleep quality, fatigue, and anxiety. arXiv preprint arXiv:2005.00096.
- Kumar, A., Shaun, M. A., and Chaurasia, B. K. (2024). Identification of psychological stress from speech signal using deep learning algorithm. *e-Prime — Advances in Electrical Engineering, Electronics and Energy*, 9:100707.

- Livingstone, S. R. and Russo, F. A. (2018). The ryerson audio-visual database of emotional speech and song (RAVDESS). *PLOS ONE*, 13(5):e0196391.
- Maslach, C. and Jackson, S. E. (1981). The measurement of experienced burnout. *Journal of Occupational Behaviour*, 2(2):99–113.
- McFee, B., Raffel, C., Liang, D., Ellis, D. P. W., McVicar, M., Battenberg, E., and Nieto, O. (2015). librosa: Audio and music signal analysis in Python. In *Proceedings of the 14th Python in Science Conference*.
- Pepino, L., Riera, P., and Ferrer, L. (2021). Emotion recognition from speech using wav2vec 2.0 embeddings. In *Proceedings of Interspeech 2021*, pages 3400–3404.
- Pradhan, S., Paul, P., and Saranya, T. S. (2024). Application of deep learning techniques to detect workplace burnout.
- Ringeval, F., Schuller, B., Valstar, M., Cowie, R., Kaya, H., Schmitt, M., and Pantic, M. (2019). AVEC 2019 workshop and challenge: State-mind-body estimation in the wild. In *Proceedings of the 9th International on Audio/Visual Emotion Challenge and Workshop*, pages 3–12.
- Russell, J. A. (1980). A circumplex model of affect. *Journal of Personality and Social Psychology*, 39(6):1161–1178.
- Schuller, B., Steidl, S., and Batliner, A. (2009). The INTERSPEECH 2009 emotion challenge. In *Proceedings of INTERSPEECH 2009*, pages 312–315.
- Schuller, B., Steidl, S., Batliner, A., Vinciarelli, A., Scherer, K., Ringeval, F., and Kim, S. (2013). The INTERSPEECH 2013 computational paralinguistics challenge: Social signals, conflict, emotion, autism. In *Proceedings of INTERSPEECH 2013*, pages 148–152.
- Trigeorgis, G., Ringeval, F., Brueckner, R., Marchi, E., Nicolaou, M. A., Schuller, B., and Zafeiriou, S. (2016). Adieu features? end-to-end speech emotion recognition using a deep convolutional recurrent network. In *Proceedings of ICASSP 2016*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.

Vekkot, S., Chavali, S. T., Kandavalli, C. T., Podila, R. S. A., Gupta, D., Zakariah, M., and Alotaibi, Y. A. (2025). Continuous speech-based fatigue detection and transition state prediction for air traffic controllers. *IEEE Access*, 13:3298–3319.

Appendix A- Sample Code

STAGE 1 MODEL SOURCE CODE

Environment Configuration

```
from pathlib import Path

# Base paths
PROJECT_ROOT      = Path(__file__).resolve().parent.parent
RAW_DATA_DIR      = PROJECT_ROOT / "data" / "raw"
PROCESSED_DATA_DIR = PROJECT_ROOT / "data" / "processed"
METADATA_DIR      = PROCESSED_DATA_DIR / "metadata"
MODELS_DIR        = PROJECT_ROOT / "models"

# Audio defaults
SAMPLE_RATE      = 16_000
N_MELS           = 80
WIN_LENGTH_MS    = 25
HOP_LENGTH_MS    = 10
```

Audio Preprocessing and Log-Mel Spectrogram Extraction

```
import librosa
import numpy as np
from .config import SAMPLE_RATE, N_MELS, WIN_LENGTH_MS, HOP_LENGTH_MS

WIN_LENGTH = int(SAMPLE_RATE * WIN_LENGTH_MS / 1000)
HOP_LENGTH = int(SAMPLE_RATE * HOP_LENGTH_MS / 1000)
MAX_DURATION_S = 3.0
MAX_SAMPLES = int(MAX_DURATION_S * SAMPLE_RATE)
_MAX_FRAMES = 1 + MAX_SAMPLES // HOP_LENGTH # ~301 frames

def loudness_norm(audio: np.ndarray,
                  eps: float = 1e-9) -> np.ndarray:
    """Normalize RMS loudness to target level 0.1."""
    rms = np.sqrt(np.mean(audio**2) + eps)
    target = 0.1
    return audio * (target / max(rms, eps))

def trim_pad(audio: np.ndarray,
             max_samples: int = MAX_SAMPLES) -> np.ndarray:
    """Trim or zero-pad audio to fixed length."""
    if len(audio) > max_samples:
        return audio[:max_samples]
    elif len(audio) < max_samples:
```

```

        return np.pad(audio, (0, max_samples - len(audio)))
    return audio

def compute_log_mel(audio: np.ndarray,
                    sr: int) -> np.ndarray:
    """Compute Log-Mel spectrogram from waveform."""
    mel = librosa.feature.melspectrogram(
        y=audio, sr=sr, n_mels=N_MELS,
        n_fft=2048, hop_length=HOP_LENGTH,
        win_length=WIN_LENGTH, center=True, power=2.0,
    )
    log_mel = librosa.power_to_db(mel, ref=np.max)
    return log_mel.astype(np.float32)

def pad_mel_to_fixed(log_mel: np.ndarray) -> np.ndarray:
    """Pad/trim mel spectrogram to exactly _MAX_FRAMES columns."""
    n_mels, t = log_mel.shape
    if t < _MAX_FRAMES:
        log_mel = np.pad(
            log_mel, ((0, 0), (0, _MAX_FRAMES - t))
        )
    elif t > _MAX_FRAMES:
        log_mel = log_mel[:, :_MAX_FRAMES]
    return log_mel

```

Dataset Loader

```

import json
import numpy as np
import torch

from torch.utils.data import Dataset
from pathlib import Path
from typing import Dict, List, Tuple
from .config import PROCESSED_DATA_DIR

POLARITY_TO_ID: Dict[str, int] = {
    "negative": 0,
    "neutral" : 1,
    "positive": 2,
}

class MelDataset(Dataset):
    def __init__(self, manifest_path: Path,
                 max_frames: int | None = None):
        self.manifest_path = manifest_path
        with manifest_path.open() as f:
            self.items: List[Dict[str, str]] = json.load(f)
        self.max_frames = max_frames

```

```

def __len__(self) -> int:
    return len(self.items)

def __getitem__(self,
                idx: int) -> Tuple[torch.Tensor, int]:
    item      = self.items[idx]
    feat_path = PROCESSED_DATA_DIR / Path(
        item["feature_path"].replace('\\', '/')
    )
    mel = np.load(feat_path)
    if self.max_frames is not None:
        mel = mel[:, :self.max_frames]
    label = POLARITY_TO_ID[item["polarity"]]
    return torch.from_numpy(mel), label

def collate_pad(batch):
    """Pad variable-length mel spectrograms to max length."""
    mels, labels = zip(*batch)
    max_len      = max(m.shape[1] for m in mels)
    padded = [
        torch.nn.functional.pad(m, (0, max_len - m.shape[1]))
        if m.shape[1] < max_len else m
        for m in mels
    ]
    return torch.stack(padded), torch.tensor(
        labels, dtype=torch.long
    )

```

CRNN-BiGRU Model Architecture

```

import torch
import torch.nn as nn

class CRNN(nn.Module):
    def __init__(self, n_mels: int = 80,
                  n_classes: int = 3,
                  hidden_size: int = 192,
                  dropout: float = 0.3):
        super().__init__()
        # Three convolutional blocks with batch normalisation
        self.cnn = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d((2, 2)),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64), nn.ReLU(),
            nn.MaxPool2d((2, 2)),
            nn.Conv2d(64, 96, kernel_size=3, padding=1),
            nn.BatchNorm2d(96), nn.ReLU(),

```

```

        nn.MaxPool2d((2, 2)),
    )
    # Bidirectional GRU over flattened time-frequency sequence
    self.gru = nn.GRU(
        input_size=(n_mels // 8) * 96,
        hidden_size=hidden_size,
        num_layers=1,
        batch_first=True,
        bidirectional=True
    )
    # Fully connected classification head
    self.head = nn.Sequential(
        nn.Linear(hidden_size * 2, hidden_size),
        nn.ReLU(),
        nn.Dropout(dropout),
        nn.Linear(hidden_size, n_classes),
    )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x.unsqueeze(1)          # (B, 1, n_mels, T)
        x = self.cnn(x)             # (B, C, n_mels/8, T/8)
        b, c, f, t = x.shape
        x = x.permute(0, 3, 1, 2).contiguous().view(
            b, t, c * f             # (B, T/8, C*f)
        )
        out, _ = self.gru(x)        # (B, T/8, 2*hidden)
        out = out[:, -1, :]         # last timestep
        return self.head(out)

```

Five-Fold Cross-Validation Training

```

import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
from torch.optim.lr_scheduler import CosineAnnealingLR
from torch.utils.data import DataLoader, Subset, WeightedRandomSampler
from collections import Counter
from .model import CRNN
from .data_loader import POLARITY_TO_ID, collate_pad
from .config import MODELS_DIR

K_FOLDS = 5

def build_class_weights(items):
    counts = Counter(it["polarity"] for it in items)
    total = sum(counts.values())
    n_classes = len(counts)
    return {lbl: total / (n_classes * cnt)

```

```

        for lbl, cnt in counts.items()

def make_weighted_sampler(items, weights_dict):
    sample_weights = [weights_dict[it["polarity"]]
                       for it in items]
    return WeightedRandomSampler(
        sample_weights,
        num_samples=len(sample_weights),
        replacement=True
    )

def kfold_split(n, k, seed=42):
    """Shuffle and split indices into k folds."""
    indices = np.arange(n)
    rng = np.random.default_rng(seed)
    rng.shuffle(indices)
    folds = np.array_split(indices, k)
    return [
        (np.concatenate([folds[j] for j in range(k) if j != i]),
         folds[i])
        for i in range(k)
    ]

def run_kfold(args):
    device = torch.device(
        "cuda" if torch.cuda.is_available() else "cpu"
    )

    all_items = load_manifest("train") + load_manifest("val")
    full_ds = ManifestItemDataset(all_items)
    splits = kfold_split(len(all_items), K_FOLDS)
    fold_accs = []

    for fold, (train_idx, val_idx) in enumerate(splits, 1):
        train_items_fold = [all_items[i] for i in train_idx]
        train_ds = Subset(full_ds, train_idx.tolist())
        val_ds = Subset(full_ds, val_idx.tolist())

        # WeightedRandomSampler for class imbalance
        weights_dict = build_class_weights(train_items_fold)
        sampler = make_weighted_sampler(
            train_items_fold, weights_dict
        )

        train_loader = DataLoader(
            train_ds, batch_size=args.batch_size,
            sampler=sampler, collate_fn=collate_pad
        )

        val_loader = DataLoader(
            val_ds, batch_size=args.batch_size,
            shuffle=False, collate_fn=collate_pad

```

```

)

# Fresh model and optimizer per fold
model = CRNN(n_mels=80, n_classes=len(POLARITY_TO_ID),
             hidden_size=args.hidden_size,
             dropout=args.dropout).to(device)

weight_tensor = torch.tensor(
    [weights_dict.get(c, 1.0)
     for c in ("negative", "neutral", "positive")],
    dtype=torch.float, device=device)
)

criterion = nn.CrossEntropyLoss(weight=weight_tensor)
optimizer = optim.Adam(model.parameters(), lr=args.lr)
scheduler = CosineAnnealingLR(
    optimizer, T_max=args.epochs, eta_min=args.lr * 0.1
)

best_val_acc = 0.0
checkpoint = MODELS_DIR / f"fold{fold}_model.pt"

for epoch in range(1, args.epochs + 1):
    train_loss, train_acc = train_one_epoch(
        model, train_loader, criterion, optimizer, device
    )
    val_loss, val_acc = evaluate(
        model, val_loader, criterion, device
    )
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), checkpoint)
    scheduler.step()

fold_accs.append(best_val_acc)
print(f"Fold {fold}: best_val_acc={best_val_acc:.4f}")

print(f"\nMean Accuracy: {np.mean(fold_accs):.4f} "
      f"+/- {np.std(fold_accs):.4f}")

```

STAGE 2 MODEL SOURCE CODE

Environment Setup and Path Configuration

```
import os
import random
import copy
import shutil
import pickle
import glob
import numpy as np
import pandas as pd

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import TensorDataset, DataLoader

import librosa
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
from scipy.stats import linregress

from sklearn.cluster import KMeans
from sklearn.preprocessing import MinMaxScaler, label_binarize
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_curve, auc, f1_score)

import warnings
warnings.filterwarnings('ignore')

# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')
print("Running in Google Colab environment.")

# Paths Configuration based on Drive Layout
base_dir = '/content/drive/MyDrive/Fatigue_Burnout_detection'
raw_data_dir = os.path.join(base_dir, 'data', 'raw')

# Create necessary directories
dirs_to_create = [
    os.path.join(base_dir, "features", "np"),
    os.path.join(base_dir, "features", "csv"),
    os.path.join(base_dir, "models", "checkpoints"),
    os.path.join(base_dir, "models", "final"),
    os.path.join(base_dir, "results", "training_curves"),
```

```

        os.path.join(base_dir, "results", "metrics")
    ]

    for d in dirs_to_create:
        os.makedirs(d, exist_ok=True)

    print(f"Base Directory set to: {base_dir}")
    print(f"Raw Data Directory set to: {raw_data_dir}")

    # Set SEED globally for reproducibility
    SEED = 42

    def set_seed(seed):
        random.seed(seed)
        np.random.seed(seed)
        torch.manual_seed(seed)
        if torch.cuda.is_available():
            torch.cuda.manual_seed(seed)
            torch.cuda.manual_seed_all(seed)
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False

    set_seed(SEED)

    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f"Using device: {device}")
    if device.type == 'cpu':
        print("WARNING: No GPU available! Training will be slow.")

```

Stage 1 Emotion Model Architecture

```

# Define Stage 1 Emotion Model Architecture (CRNN)
class CRNN(nn.Module):
    def __init__(self, n_mels=80, n_classes=3,
                  hidden_size=192, dropout=0.3):
        super().__init__()
        self.cnn = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d((2, 2)),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d((2, 2)),
            nn.Conv2d(64, 96, kernel_size=3, padding=1),
            nn.BatchNorm2d(96), nn.ReLU(), nn.MaxPool2d((2, 2)),
        )
        self.gru = nn.GRU(
            input_size=(n_mels // 8) * 96,
            hidden_size=hidden_size,
            num_layers=1,
            batch_first=True,
            bidirectional=True
        )
        self.head = nn.Sequential(

```

```

        nn.Linear(hidden_size * 2, hidden_size),
        nn.ReLU(),
        nn.Dropout(dropout),
        nn.Linear(hidden_size, n_classes),
    )

    def forward(self, x):
        x = x.unsqueeze(1)
        x = self.cnn(x)
        b, c, f, t = x.shape
        x = x.permute(0, 3, 1, 2).contiguous().view(b, t, c * f)
        out, _ = self.gru(x)
        out = out[:, -1, :]
        return self.head(out)

# Load pre-trained emotion model
emotion_model_path = (
    '/content/drive/MyDrive/VoiceAnalysis DL+GRU/crnn_best.pt'
)

try:
    emotion_model = CRNN(n_classes=3).to(device)
    emotion_model.load_state_dict(
        torch.load(emotion_model_path, map_location=device)
    )
    emotion_model.eval()
    for param in emotion_model.parameters():
        param.requires_grad = False
    print("CRNN Model successfully loaded!")
except Exception as e:
    print(f"Error loading model: {e}")

def get_emotion_probs(audio_path, model, device):
    y, sr = librosa.load(audio_path, sr=16000, mono=True)
    y, _ = librosa.effects.trim(y, top_db=20)
    mel_spec = librosa.feature.melspectrogram(
        y=y, sr=sr, n_fft=1024, hop_length=512, n_mels=80
    )
    mel_spec_db = librosa.power_to_db(mel_spec, ref=np.max)
    input_tensor = torch.tensor(
        mel_spec_db, dtype=torch.float32
    ).unsqueeze(0).to(device)
    with torch.no_grad():
        logits = model(input_tensor)
        probs = F.softmax(logits, dim=1).cpu().numpy()[0]
    return probs[0], probs[1], probs[2]

```

Acoustic Feature Extraction and Arousal Computation

```
def extract_acoustic_features(audio_path):
    y, sr = librosa.load(audio_path, sr=16000, mono=True)
    y, _ = librosa.effects.trim(y, top_db=20)

    rms_mean = np.mean(librosa.feature.rms(y=y)[0])

    f0, voiced_flag, voiced_probs = librosa.pyin(
        y, fmin=75, fmax=300, sr=sr
    )
    f0_voiced = f0[~np.isnan(f0)]
    f0_mean = np.mean(f0_voiced) if len(f0_voiced) > 0 else 0
    f0_var = np.var(f0_voiced) if len(f0_voiced) > 0 else 0

    centroid_mean = np.mean(
        librosa.feature.spectral_centroid(y=y, sr=sr)[0]
    )
    rolloff_mean = np.mean(
        librosa.feature.spectral_rolloff(
            y=y, sr=sr, roll_percent=0.85
        )[0]
    )
    zcr_mean = np.mean(
        librosa.feature.zero_crossing_rate(y=y)[0]
    )
    return [rms_mean, f0_mean, f0_var,
            centroid_mean, rolloff_mean, zcr_mean]

# Compute arousal and fatigue scores
# Global Scaler (for pseudo-labelling only, not cross-validation)
global_scaler = MinMaxScaler()
raw_acoustic_norm = global_scaler.fit_transform(raw_acoustic)

A_acoustic = (0.30 * raw_acoustic_norm[:, 0] +
              0.25 * raw_acoustic_norm[:, 1] +
              0.15 * raw_acoustic_norm[:, 2] +
              0.15 * raw_acoustic_norm[:, 5] +
              0.10 * raw_acoustic_norm[:, 3] +
              0.05 * raw_acoustic_norm[:, 4])

A_emotion = (0.8 * emotion_probs[:, 0] +
             0.2 * emotion_probs[:, 1] +
             0.4 * emotion_probs[:, 2])

alpha = 0.60
A_final = alpha * A_emotion + (1 - alpha) * A_acoustic

beta = 0.70
F_score = beta * (1 - A_final) + (1 - beta) * emotion_probs[:, 1]
```

```

# K-Means clustering for pseudo-label generation
kmeans = KMeans(n_clusters=3, random_state=SEED, n_init=10)
labels_raw = kmeans.fit_predict(F_score.reshape(-1, 1))

order = np.argsort(kmeans.cluster_centers_.sum(axis=1))
mapping = {order[0]: 0, order[1]: 1, order[2]: 2}
mapping_str = {
    0: "Non-Fatigued",
    1: "Mild Fatigue",
    2: "High Fatigue / Burnout"
}
pseudo_labels = np.array([mapping[l] for l in labels_raw])
pseudo_label_str = [mapping_str[l] for l in pseudo_labels]

```

Temporal Window Construction

```

W = 10 # Window size
S = 5 # Stride
gamma = 0.30

window_features = []
raw_sequences_list = []
emotion_seqs_list = []
temporal_labels = []
global_scaled_sequences = []

for speaker in features_df['speaker_id'].unique():
    spk_df = features_df[
        features_df['speaker_id'] == speaker
    ].sort_values('file')
    if len(spk_df) < W:
        continue

    for i in range(0, len(spk_df) - W + 1, S):
        window = spk_df.iloc[i:i+W]

        A_fin_seq = window['A_final'].values
        F_score_seq = window['F_score'].values

        slope, _, _, _, _ = linregress(range(W), A_fin_seq)
        if np.isnan(slope):
            slope = 0

        win_label = int(window['Pseudo_Label'].values.max())

        raw_ac = window[['RMS_mean', 'F0_mean', 'F0_var',
                        'Centroid_mean', 'Rolloff_mean',
                        'ZCR_mean']].values

```

```

emo_arr = window[['P_pos', 'P_neg', 'P_neu']].values

raw_sequences_list.append(raw_ac)
emotion_seqs_list.append(emo_arr)
temporal_labels.append(win_label)

# Build globally scaled 3D sequence (shape: W x 10)
A_em_arr = window['A_emotion'].values.reshape(-1, 1)
A_fin_arr = A_fin_seq.reshape(-1, 1)
F_sc_arr = F_score_seq.reshape(-1, 1)
slope_arr = np.full((W, 1), slope)

scaled_ac = global_scaler.transform(raw_ac)
seq_3d = np.concatenate(
    [scaled_ac, A_em_arr, A_fin_arr, F_sc_arr, slope_arr],
    axis=1
)
global_scaled_sequences.append(seq_3d)

X_raw_np = np.array(raw_sequences_list)
X_emo_np = np.array(emotion_seqs_list)
y_np = np.array(temporal_labels)

```

Stage 2 BiGRU-Attention Model and Cross-Validation Training

```

# BiGRU-Attention Model Definition
class BiGRUAttention(nn.Module):
    def __init__(self, input_dim=10, hidden_dim=64,
                  num_layers=2, num_classes=3, dropout=0.3):
        super().__init__()
        self.bigru = nn.GRU(
            input_size=input_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True,
            bidirectional=True,
            dropout=dropout if num_layers > 1 else 0
        )
        self.attn_q = nn.Linear(hidden_dim * 2, hidden_dim * 2)
        self.attn_k = nn.Linear(hidden_dim * 2, hidden_dim * 2)
        self.attn_v = nn.Linear(hidden_dim * 2, hidden_dim * 2)
        self.head = nn.Sequential(
            nn.Linear(hidden_dim * 2, hidden_dim),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim, num_classes)
        )

```

```

def forward(self, x):
    h, _ = self.bigru(x)
    Q = self.attn_q(h)
    K = self.attn_k(h)
    V = self.attn_v(h)
    d_k = Q.size(-1)
    scores = torch.bmm(Q, K.transpose(1, 2)) / (d_k ** 0.5)
    weights = F.softmax(scores, dim=-1)
    context = torch.bmm(weights, V)
    out = context[:, -1, :]
    return self.head(out)

# Helper to build per-fold input tensors
def build_X_split(ac_scaled, emo):
    N = ac_scaled.shape[0]
    out = np.zeros((N, W, 10))
    out[:, :, 0:6] = ac_scaled

    A_ac = (0.3*ac_scaled[:, :, 0] + 0.25*ac_scaled[:, :, 1] +
            0.15*ac_scaled[:, :, 2] + 0.15*ac_scaled[:, :, 5] +
            0.10*ac_scaled[:, :, 3] + 0.05*ac_scaled[:, :, 4])
    A_em = (0.8*emo[:, :, 0] + 0.2*emo[:, :, 1] + 0.4*emo[:, :, 2])
    A_fn = alpha * A_em + (1 - alpha) * A_ac
    F_sn = beta * (1 - A_fn) + (1 - beta) * emo[:, :, 1]

    out[:, :, 6] = A_em
    out[:, :, 7] = A_fn
    out[:, :, 8] = F_sn

    for b in range(ac_scaled.shape[0]):
        s, _, _, _, _ = linregress(range(W), A_fn[b])
        out[b, :, 9] = s if not np.isnan(s) else 0

    return torch.tensor(out, dtype=torch.float32)

# Five-Fold Stratified Cross-Validation
skf = StratifiedKFold(n_splits=5, shuffle=True,
                      random_state=SEED)

epochs = 100
batch_size = 64
patience = 10

fold_f1_scores = []
fold_val_preds_probs = {}
fold_val_labels = {}
cv_records = []

for fold, (train_idx, val_idx) in enumerate(
    skf.split(X_raw_np, y_np)):

```

```

print(f"\n--- Fold {fold+1} ---")

train_raw = X_raw_np[train_idx]
val_raw    = X_raw_np[val_idx]
train_emo  = X_emo_np[train_idx]
val_emo    = X_emo_np[val_idx]

# Fit scaler on training fold only (prevents data leakage)
scaler_fold = MinMaxScaler()
scaler_fold.fit(train_raw.reshape(-1, 6))
if fold == 0:
    with open(os.path.join(npy_dir, "feature_scaler.pkl"),
              "wb") as f:
        pickle.dump(scaler_fold, f)

train_scaled = scaler_fold.transform(
    train_raw.reshape(-1, 6)
).reshape(train_raw.shape)
val_scaled = scaler_fold.transform(
    val_raw.reshape(-1, 6)
).reshape(val_raw.shape)

X_train_tsr = build_X_split(train_scaled, train_emo)
X_val_tsr   = build_X_split(val_scaled, val_emo)
y_train_tsr = torch.tensor(y_np[train_idx], dtype=torch.long)
y_val_tsr   = torch.tensor(y_np[val_idx], dtype=torch.long)

# Class-weighted loss per fold
class_counts_fold = np.bincount(y_np[train_idx], minlength=3)
class_counts_fold = np.where(class_counts_fold == 0, 1,
                             class_counts_fold)
class_weights_fold = len(train_idx) / (3.0 * class_counts_fold)
class_weights_tsr = torch.tensor(
    class_weights_fold, dtype=torch.float32
).to(device)
criterion = nn.CrossEntropyLoss(weight=class_weights_tsr)

train_loader = DataLoader(
    TensorDataset(X_train_tsr, y_train_tsr),
    batch_size=batch_size, shuffle=True
)
val_loader = DataLoader(
    TensorDataset(X_val_tsr, y_val_tsr),
    batch_size=batch_size, shuffle=False
)

model = BiGRUAttention(input_dim=10,
                       num_classes=3).to(device)
optimizer = torch.optim.AdamW(

```

```

        model.parameters(), lr=1e-3, weight_decay=1e-4
    )

    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode='min', patience=5, factor=0.5
    )

    best_val_loss = float('inf')
    best_model_weights = None
    patience_counter = 0

    for epoch in range(epochs):
        # Training loop
        model.train()
        train_loss, correct, total = 0, 0, 0
        for X_b, y_b in train_loader:
            X_b, y_b = X_b.to(device), y_b.to(device)
            optimizer.zero_grad()
            out = model(X_b)
            loss = criterion(out, y_b)
            loss.backward()
            optimizer.step()
            train_loss += loss.item() * X_b.size(0)
            _, preds = torch.max(out, 1)
            correct += (preds == y_b).sum().item()
            total += y_b.size(0)

        # Validation loop
        model.eval()
        val_loss, correct, total = 0, 0, 0
        all_val_preds, all_val_labels_ep = [], []
        with torch.no_grad():
            for X_b, y_b in val_loader:
                X_b, y_b = X_b.to(device), y_b.to(device)
                out = model(X_b)
                loss = criterion(out, y_b)
                val_loss += loss.item() * X_b.size(0)
                _, preds = torch.max(out, 1)
                correct += (preds == y_b).sum().item()
                total += y_b.size(0)
                all_val_preds.extend(preds.cpu().numpy())
                all_val_labels_ep.extend(y_b.cpu().numpy())

        val_loss_epoch = val_loss / total
        scheduler.step(val_loss_epoch)

        # Early stopping on validation loss
        if val_loss_epoch < best_val_loss:
            best_val_loss = val_loss_epoch
            best_model_weights = copy.deepcopy(model.state_dict())

```

```

        torch.save(
            best_model_weights,
            os.path.join(base_dir, "models", "checkpoints",
                          f"fold_{fold+1}_best.pt")
        )
        patience_counter = 0
    else:
        patience_counter += 1
    if patience_counter >= patience:
        print(f"Early stopping at epoch {epoch+1}")
        model.load_state_dict(best_model_weights)
        break

print("Cross Validation Complete.")

```

Alpha Sensitivity Analysis

```

def run_alpha_sensitivity():
    print("\nRunning Alpha Sensitivity Analysis (3-Fold)")
    alphas = [0.3, 0.4, 0.5, 0.6, 0.7]
    alpha_macro_f1_list = []

    for a in alphas:
        # Recompute A_final with new alpha
        A_fin = a * A_emotion + (1 - a) * A_acoustic
        F_sc = (beta * (1 - A_fin) +
                (1 - beta) * emotion_probs[:, 1])

        km = KMeans(n_clusters=3, random_state=SEED,
                    n_init=10)
        l_raw = km.fit_predict(F_sc.reshape(-1, 1))
        o = np.argsort(km.cluster_centers_.sum(axis=1))
        m = {o[0]: 0, o[1]: 1, o[2]: 2}
        temp_labels = np.array([m[l] for l in l_raw])
        features_df['temp_L_sens'] = temp_labels

        # Rebuild window labels
        win_labels = []
        for speaker in features_df['speaker_id'].unique():
            spk_df = features_df[
                features_df['speaker_id'] == speaker
            ].sort_values('file')
            if len(spk_df) < W:
                continue
            for i in range(0, len(spk_df) - W + 1, S):
                win = spk_df.iloc[i:i+W]
                maj = np.bincount(
                    win['temp_L_sens'].values.astype(int)
                ).argmax()

```

```

win_labels.append(maj)

y_alpha_np = np.array(win_labels)
if len(y_alpha_np) < 3:
    alpha_macro_f1_list.append(0)
    continue

skf3 = StratifiedKFold(n_splits=3, shuffle=True,
                        random_state=SEED)

fold_f1 = []

for train_idx, val_idx in skf3.split(X_raw_np,
                                     y_alpha_np):

    sf = MinMaxScaler()
    sf.fit(X_raw_np[train_idx].reshape(-1, 6))
    t_scaled = sf.transform(
        X_raw_np[train_idx].reshape(-1, 6)
    ).reshape(X_raw_np[train_idx].shape)
    v_scaled = sf.transform(
        X_raw_np[val_idx].reshape(-1, 6)
    ).reshape(X_raw_np[val_idx].shape)

    X_t = build_X_split(t_scaled,
                        X_emo_np[train_idx])
    X_v = build_X_split(v_scaled,
                        X_emo_np[val_idx])
    Y_t = torch.tensor(y_alpha_np[train_idx],
                        dtype=torch.long)
    Y_v = torch.tensor(y_alpha_np[val_idx],
                        dtype=torch.long)

    md = BiGRUAttention(input_dim=10,
                        num_classes=3).to(device)
    opt = torch.optim.AdamW(md.parameters(), lr=1e-3)
    crit = nn.CrossEntropyLoss()
    train_ldr = DataLoader(
        TensorDataset(X_t, Y_t),
        batch_size=64, shuffle=True
    )

    best_l, best_wt, pat_c = float('inf'), None, 0
    for ep in range(30):
        md.train()
        for xb, yb in train_ldr:
            xb, yb = xb.to(device), yb.to(device)
            opt.zero_grad()
            crit(md(xb), yb).backward()
            opt.step()
        md.eval()

```

```

        with torch.no_grad():
            vl = crit(
                md(X_v.to(device)), Y_v.to(device)
            ).item()
        if vl < best_l:
            best_l = vl
            best_wt = copy.deepcopy(md.state_dict())
            pat_c = 0
        else:
            pat_c += 1
            if pat_c >= 5:
                break

    if best_wt:
        md.load_state_dict(best_wt)
    md.eval()
    with torch.no_grad():
        _, pr = torch.max(md(X_v.to(device)), 1)
        fold_f1.append(f1_score(
            Y_v.numpy(), pr.cpu().numpy(),
            average='macro', zero_division=0
        ))

    mean_f1 = np.mean(fold_f1)
    alpha_macro_f1_list.append(mean_f1)
    print(f"Alpha {a} -> Mean Macro F1: {mean_f1:.4f}")

try:
    run_alpha_sensitivity()
except Exception as e:
    print(f"Alpha sensitivity error: {e}")

```

Inference Function

```

def predict_fatigue(audio_paths, model, emotion_model,
                    scaler, W=10):
    if len(audio_paths) != W:
        raise ValueError(
            f"Expected W={W} audio paths."
        )
    seq_ac, seq_emo = [], []
    for p in audio_paths:
        feats = extract_acoustic_features(p)
        p_pos, p_neg, p_neu = get_emotion_probs(
            p, emotion_model, device
        )
        seq_ac.append(feats)
        seq_emo.append([p_pos, p_neg, p_neu])

```

```

seq_ac = np.array(seq_ac)    # (W, 6)
seq_emo = np.array(seq_emo)  # (W, 3)

scaled_ac = scaler.transform(seq_ac)
A_ac = (0.3*scaled_ac[:,0] + 0.25*scaled_ac[:,1] +
        0.15*scaled_ac[:,2] + 0.15*scaled_ac[:,5] +
        0.10*scaled_ac[:,3] + 0.05*scaled_ac[:,4])
A_em = (0.8*seq_emo[:,0] + 0.2*seq_emo[:,1] +
        0.4*seq_emo[:,2])
A_fin = alpha * A_em + (1 - alpha) * A_ac
F_sc = (beta * (1 - A_fin) +
        (1 - beta) * seq_emo[:,1])

slope, _, _, _, _ = linregress(range(W), A_fin)
if np.isnan(slope):
    slope = 0

X_infer = np.zeros((1, W, 10))
X_infer[0, :, 0:6] = scaled_ac
X_infer[0, :, 6] = A_em
X_infer[0, :, 7] = A_fin
X_infer[0, :, 8] = F_sc
X_infer[0, :, 9] = slope

X_tsr = torch.tensor(
    X_infer, dtype=torch.float32
).to(device)
model.eval()
with torch.no_grad():
    logits = model(X_tsr)
    probs = F.softmax(logits, dim=1).cpu().numpy()[0]
    label_pred = int(np.argmax(probs))

labels = {
    0: "Non-Fatigued",
    1: "Mild Fatigue",
    2: "High Fatigue / Burnout"
}

return {
    "fatigue_score" : float(
        np.mean(F_sc) + gamma * max(0, -slope)
    ),
    "label" : label_pred,
    "label_str" : labels[label_pred],
    "class_probs" : probs.tolist(),
    "arousal_trend" : float(slope)
}

```

Appendix B Publication details

- Abineshkumar S & Gayathri S. (2026). Speech-Based Fatigue and Early Burnout Prediction Using Arousal-Informed Temporal Modelling with Bidirectional GRU. *Proceedings of ICAEM 2026 - International Conference on Advanced Engineering and Management*. (Accepted for publication)